Java basics

Programming in Java

Elementary programming, selections, characters and strings, loops, methods, arrays, recursion

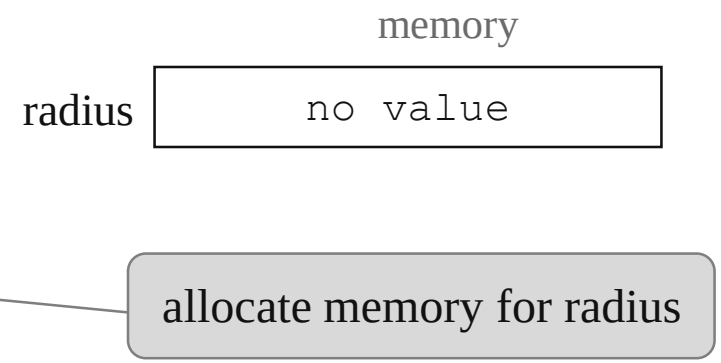
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



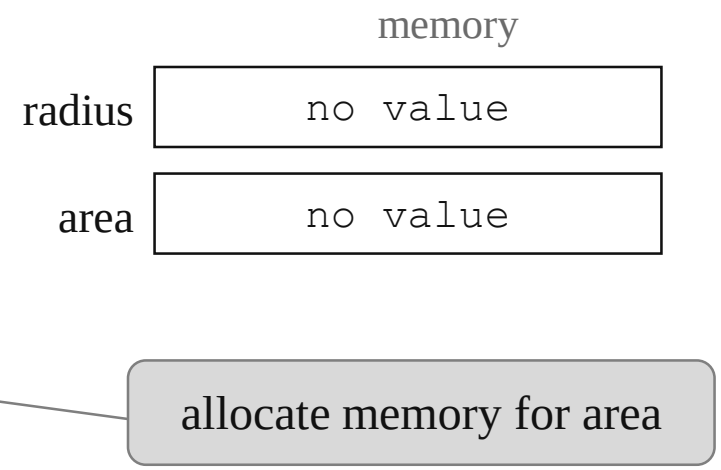
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



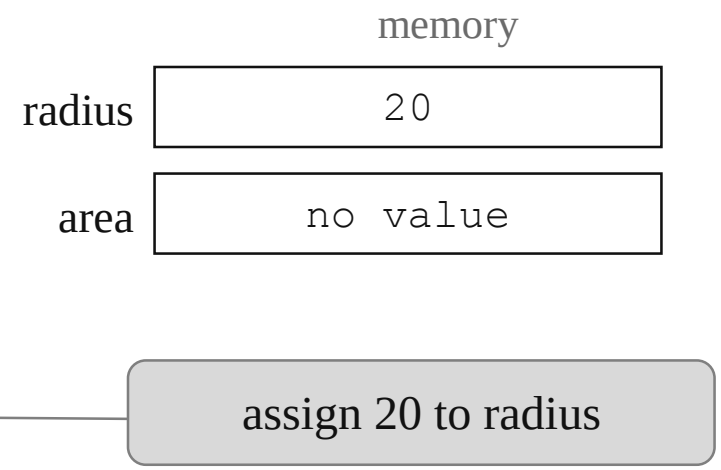
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



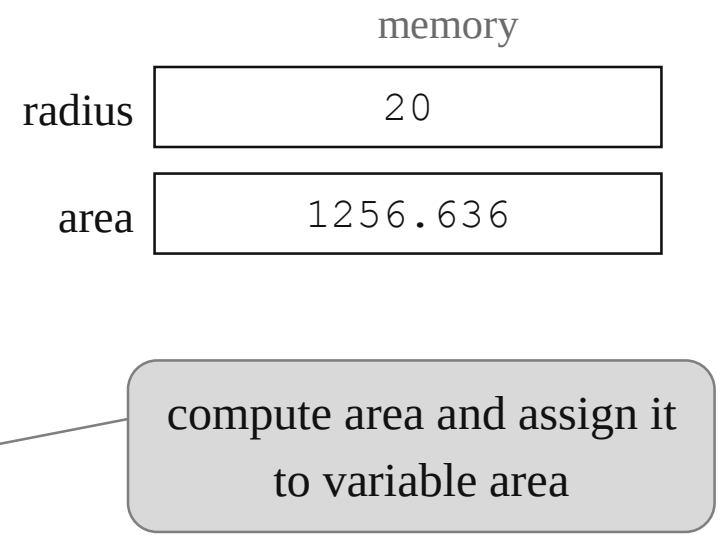
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```

memory

radius	20
area	1256.636

print a message to the console



Reading Input from the Console

1. Create a Scanner object

```
import java.util.Scanner;   // at the top of the file
Scanner input = new Scanner(System.in);
```

2. Use the methods **`next()`**, **`nextByte()`**, **`nextShort()`**, **`nextInt()`**, **`nextLong()`**, **`nextFloat()`**, **`nextDouble()`**, or **`nextBoolean()`** to obtain a string, byte, short, int, long, float, double, or boolean value. For example,

```
System.out.print("Enter a double value: ");
Scanner input = new Scanner(System.in);
double d = input.nextDouble();
```

▸ ComputeAreaWithConsoleInput ▸ ComputeAverage

Identifiers

- An identifier is a sequence of characters that consist of letters, digits, underscores (_), and dollar signs ($).
- An identifier must start with a letter, an underscore (_), or a dollar sign ($). It cannot start with a digit.
  - An identifier cannot be a reserved word.
- An identifier cannot be **`true`**, **`false`**, or **`null`**.
- An identifier can be of any length.

Variables

```
// Compute the first area
radius = 1.0;
area = radius * radius * 3.14159;
System.out.println("The area is " + area + " for radius "+radius);

// Compute the second area
radius = 2.0;
area = radius * radius * 3.14159;
System.out.println("The area is " + area + " for radius "+radius);
```

Declaring Variables and Assignment Statements

Declaring variables

```
int x;          // Declare x to be an
                // integer variable;
double radius; // Declare radius to
                // be a double variable;
char a;         // Declare a to be a
                // character variable;
```

Assignment statements

```
x = 1;           // Assign 1 to x;
radius = 1.0;    // Assign 1.0 to radius;
a = 'A';         // Assign 'A' to a;
```

Declaring and initializing in one step

```
int x = 1;
double d = 1.4;
```

Constants

```
final datatype CONSTANTNAME = VALUE;

final double PI = 3.14159;
final int SIZE = 3;
```

Numerical Data Types

Name	Range	Storage Size
byte	-2^7 (-128) to 2^7-1 (127)	8-bit signed
short	-2^{15} (-32768) to $2^{15}-1$ (32767)	16-bit signed
int	-2^{31} (-2147483648) to $2^{31}-1$ (2147483647)	32-bit signed
long	-2^{63} to $2^{63}-1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754

Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2

Integer Division

`+`, `-`, `*`, `/`, and `%`

`5 / 2` yields an integer 2.

`5.0 / 2` yields a double value 2.5

`5 % 2` yields 1 (the remainder of the division)

NOTE

Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy. For example,

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

displays 0.5000000000000001, not 0.5, and

```
System.out.println(1.0 - 0.9);
```

displays 0.09999999999999998, not 0.1. Integers are stored precisely.

Therefore, calculations with integers yield a precise integer result.

Number Literals

A literal is a constant value that appears directly in the program.
For example, 34, 1,000,000, and 5.0 are literals in the following statements:

```
int i = 34;
long x = 1000000;
double d = 5.0;
```

Integer Literals

An integer literal can be assigned to an integer variable as long as it can fit into the variable. A compilation error would occur if the literal were too large for the variable to hold. For example, the statement **`byte b = 1000`** would cause a compilation error, because 1000 cannot be stored in a variable of the **`byte`** type.

An integer literal is assumed to be of the **`int`** type, whose value is between -2^{31} (-2147483648) to $2^{31}-1$ (2147483647). To denote an integer literal of the **`long`** type, append it with the letter L or l. L is preferred because l (lowercase L) can easily be confused with 1 (the digit one).

Floating-Point Literals

Floating-point literals are written with a decimal point. By default, a floating-point literal is treated as a **`double`** type value. For example, 5.0 is considered a **`double`** value, not a **`float`** value. You can make a number a **`float`** by appending the letter f or F, and make a number a **`double`** by appending the letter d or D. For example, you can use 100.2f or 100.2F for a float number, and 100.2d or 100.2D for a double number.

Scientific Notation

Floating-point literals can also be specified in scientific notation, for example, 1.23456e+2, same as 1.23456e2, is equivalent to 123.456, and 1.23456e-2 is equivalent to 0.0123456. E (or e) represents an exponent and it can be either in lowercase or uppercase.

Numeric Type Conversion

When performing a binary operation involving two operands of different types, Java automatically converts the operand based on the following rules:

1. If one of the operands is double, the other is converted into double.
2. Otherwise, if one of the operands is float, the other is converted into float.
3. Otherwise, if one of the operands is long, the other is converted into long.
4. Otherwise, both operands are converted into int.

Type Casting

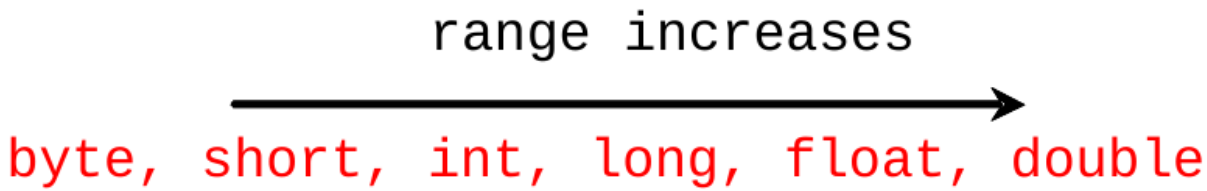
Implicit casting

```
double d = 3; (type widening)
```

Explicit casting

```
int i = (int)3.0; (type narrowing)
int i = (int)3.9; (Fraction part is truncated)
```

What is wrong? `int x = 5 / 2.0;`



Debugging

Logic errors are called bugs. The process of finding and correcting errors is called **debugging**. A common approach to debugging is to use a combination of methods to narrow down to the part of the program where the bug is located. You can hand-trace the program (i.e., catch errors by reading the program), or you can insert print statements in order to show the values of the variables or the execution flow of the program. This approach might work for a short, simple program. But for a large, complex program, the most effective approach for debugging is to use a debugger utility.

Debugger

Debugger is a program that facilitates debugging. You can use a debugger to

- Execute a single statement at a time.
- Trace into or stepping over a method.
- Set breakpoints.
- Display variables.
- Display call stack.
- Modify variables.

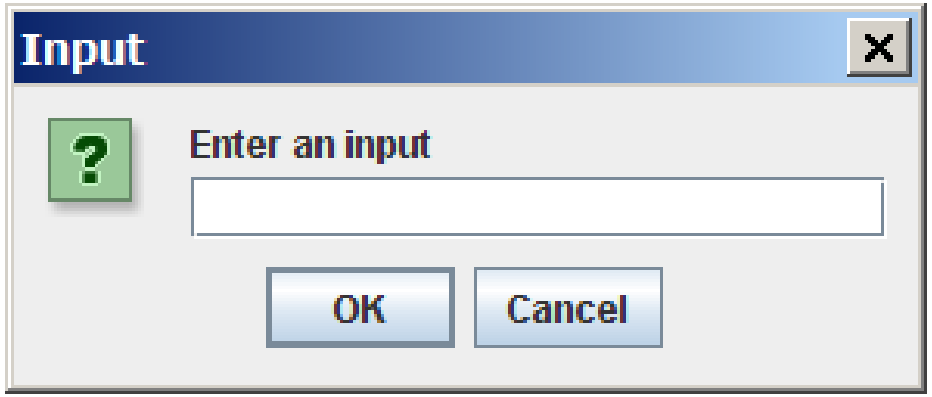
JOptionPane Input

Two ways of obtaining input.

1. Using the Scanner class (console input)
2. Using JOptionPane input dialogs

Getting Input from Input Dialog Boxes

```
import javax.swing.JOptionPane;
String input = JOptionPane.showInputDialog(
  "Enter an input");
```



Getting Input from Input Dialog Boxes

```
String string = JOptionPane.showInputDialog(
  null, "Prompting Message", "Dialog Title",
  JOptionPane.QUESTION_MESSAGE);
```



Two Ways to Invoke the Method

There are several ways to use the **`showInputDialog`** method. For the time being, you only need to know two ways to invoke it.

One is to use a statement as shown in the example:

```
String string = JOptionPane.showInputDialog(null, x,
  y, JOptionPane.QUESTION_MESSAGE);
```

where x is a string for the prompting message, and y is a string for the title of the input dialog box.

The other is to use a statement like this:

```
JOptionPane.showInputDialog(x);
```

where x is a string for the prompting message.

Converting Strings to Integers and Doubles

The input returned from the input dialog box is a string. If you enter a numeric value such as 123, it returns “123”. To obtain the input as a number, you have to convert a string into a number.

To convert a string into an **`int`** value, you can use the static **`parseInt`** method in the **`Integer`** class as follows:

```
int intValue = Integer.parseInt(intString);
```

where **`intString`** is a numeric string such as “123”.

To convert a string into a **`double`** value, you can use the static **`parseDouble`** method in the **`Double`** class as follows:

```
double doubleValue = Double.parseDouble(doubleString);
```

where **`doubleString`** is a numeric string such as “123.45”.

Java basics

Selections

The boolean Type and Operators

Often in a program you need to compare two values, such as whether i is greater than j. Java provides six comparison operators (also known as relational operators) that can be used to compare two values. The result of the comparison is a Boolean value: true or false.

```
boolean b = (1 > 2);
```

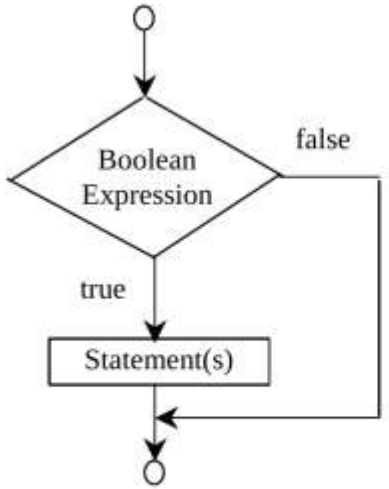
Comparison Operators

Operator	Name
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

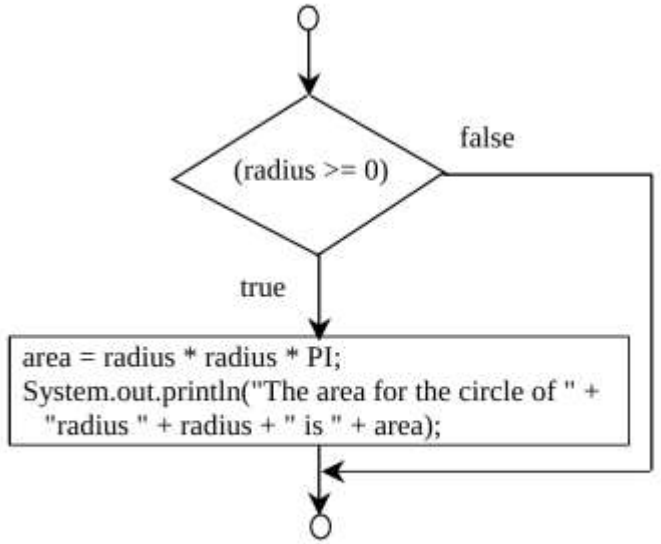
One-way `if` Statements

```
if (boolean-expression) {
  statement(s);
}
```

```
if (radius >= 0) {
  area = radius * radius * PI;
  System.out.println("The area"
    + " for the circle of radius "
    + radius + " is " + area);
}
```



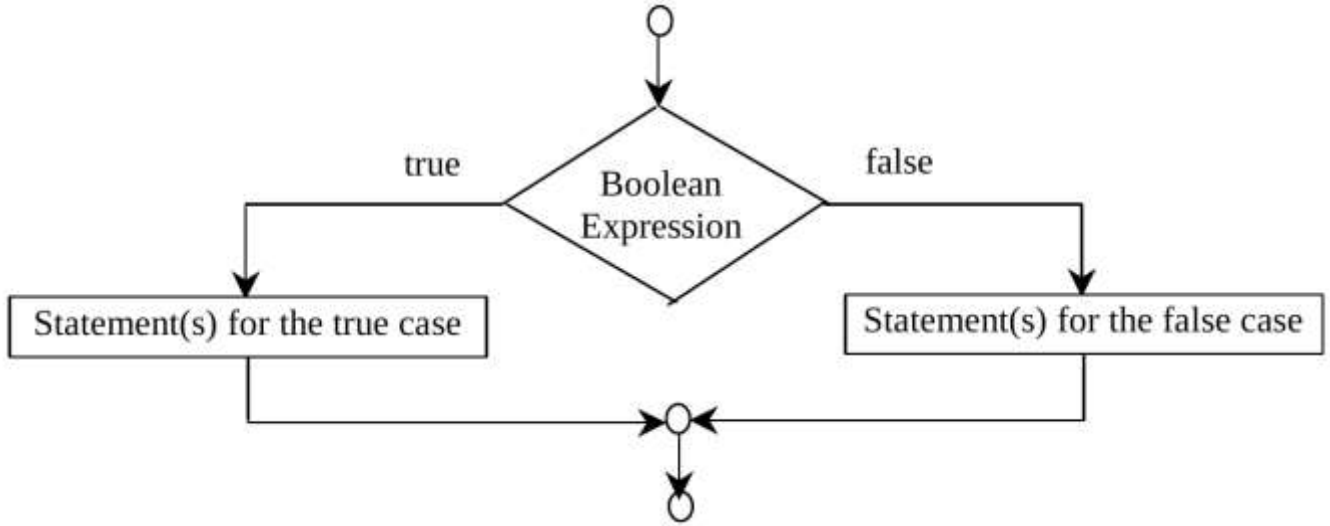
(A)



(B)

The Two-way `if` Statement

```
if (boolean-expression) {
  statement(s)-for-the-true-case;
}
else {
  statement(s)-for-the-false-case;
}
```



`if...else` Example

```
if (radius >= 0) {
  area = radius * radius * 3.14159;

  System.out.println("The area for the "
    + "circle of radius " + radius +
    " is " + area);
}
else {
  System.out.println("Negative input");
}
```

Note

- The **`else`** clause matches the most recent **`if`** clause in the same block.

```
int i = 1;
int j = 2;
int k = 3;

if (i > j)
  if (i > k)
    System.out.println("A");
else
    System.out.println("B");
```

(a)

Equivalent

```
int i = 1;
int j = 2;
int k = 3;

if (i > j)
  if (i > k)
    System.out.println("A");
  else
    System.out.println("B");
```

(b)

Note, cont.

Nothing is printed from the preceding statement. To force the **`else`** clause to match the first **`if`** clause, you must add a pair of braces:

```
int i = 1;
int j = 2;
int k = 3;
if (i > j) {
  if (i > k)
    System.out.println("A");
}
else
  System.out.println("B");
```

This statement prints B.

Common Errors

Adding a semicolon at the end of an **`if`** clause is a common mistake.

```
if (radius >= 0);
{
  area = radius*radius*PI;
  System.out.println(
    "The area for the circle of radius " +
    radius + " is " + area);
}
```

This mistake is hard to find, because it is not a compilation error or a runtime error, it is a logic error.

This error often occurs when you use the next-line block style.

TIP

```
if (number % 2 == 0)
  even = true;
else
  even = false;
```

(a)

Equivalent

```
boolean even
  = number % 2 == 0;
```

(b)

CAUTION

```
if (even == true)
  System.out.println(
    "It is even.");
```

(a)

Equivalent

```
if (even)
  System.out.println(
    "It is even.");
```

(b)

Logical Operators

Operator	Name
`!`	`not`
`&&`	`and`
`||`	`or`
`^`	`exclusive or`

Truth Table for Operator !

p	!p
true	false
false	true

Example (assume age = 24, gender = 'M')

!(age > 18) is false, because (age > 18) is true.

!(gender != 'F') is true, because (grade != 'F') is false.

Truth Table for Operator &&

p1	p2	p1 && p2
false	false	false
false	true	false
true	false	false
true	true	true

Example (assume age = 24, gender = 'F')

(age > 18) && (gender == 'F') is true, because (age > 18) and (gender == 'F') are both true.

(age > 18) && (gender != 'F') is false, because (gender != 'F') is false.

Truth Table for Operator ||

p1	p2	p1 || p2
false	false	false
false	true	true
true	false	true
true	true	true

Example (assume age = 24, gender = 'F')

(age > 34) || (gender == 'F') is true, because (gender == 'F') is true.

(age > 34) || (gender == 'M') is false, because (age > 34) and (gender == 'M') are both false.

Truth Table for Operator ^

p1	p2	p1 ^ p2
false	false	false
false	true	true
true	false	true
true	true	false

Example (assume age = 24, gender = 'F')

(age > 34) ^ (gender == 'F') is true, because (age > 34) is false but (gender == 'F') is true.

(age > 34) || (gender == 'M') is false, because (age > 34) and (gender == 'M') are both false.

Examples

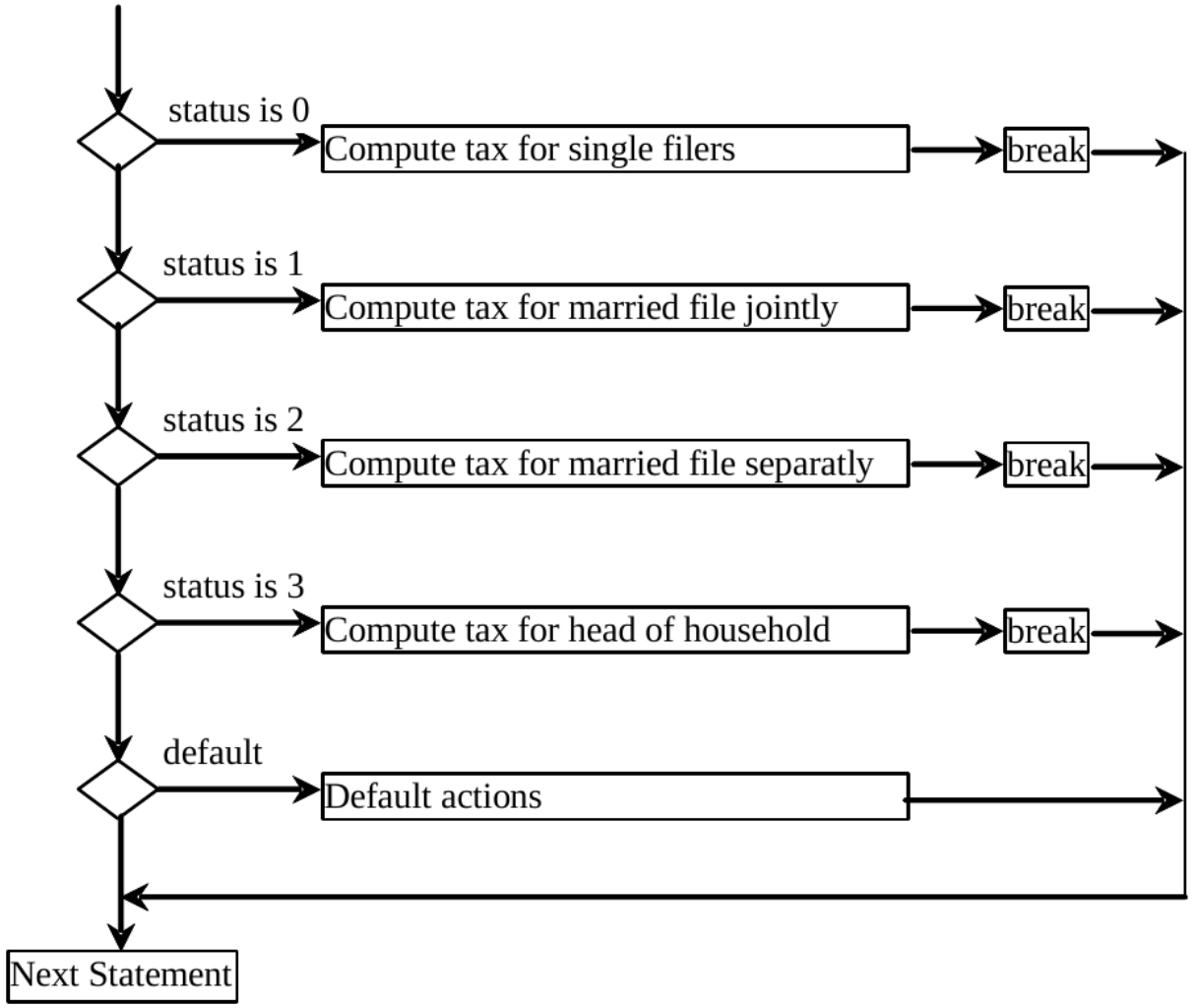
```
System.out.println("Is " + number + " divisible by 2 and 3? " +
  ((number % 2 == 0) && (number % 3 == 0)));

System.out.println("Is " + number + " divisible by 2 or 3? " +
  ((number % 2 == 0) || (number % 3 == 0)));

System.out.println("Is " + number +
  " divisible by 2 or 3, but not both? " +
  ((number % 2 == 0) ^ (number % 3 == 0)));
```

▶ TestBooleanOperators

`switch` Statement Flow Chart



`switch` Statement Rules

The switch-expression must yield a value of char, byte, short, int, String or enum type and must always be enclosed in parentheses.

The value1, ..., and valueN must have the same data type as the value of the switch-expression. The resulting statements in the case statement are executed when the value in the case statement matches the value of the switch-expression. Note that value1, ..., and valueN are constant expressions, meaning that they cannot contain variables in the expression, such as 1 + x.

```
switch (switch-expression) {
  case value1: statement(s)1;
               break;
  case value2: statement(s)2;
               break;
  ...
  case valueN: statement(s)N;
               break;
  default: statement(s)-for-default;
}
```

`switch` Statement Rules

The keyword break is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.

The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.

```
switch (switch-expression) {
  case value1: statement(s)1;
               break;
  case value2: statement(s)2;
               break;
  ...
  case valueN: statement(s)N;
               break;
  default: statement(s)-for-default;
}
```

The case statements are executed in sequential order, but the order of the cases (including the default case) does not matter. However, it is good programming style to follow the logical sequence of the cases and place the default case at the end.

Trace `switch` statement

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

```
switch (ch) {
  case 'a': System.out.println(ch);
            break;
  case 'b': System.out.println(ch);
            break;
  case 'c': System.out.println(ch);
}
```

Conditional Operator

```
if (x > 0)
  y = 1
else
  y = -1;
```

is equivalent to

```
y = (x > 0) ? 1 : -1;
```

```
(boolean-expression) ? expression1 : expression2
```

Ternary operator

Binary operator

Unary operator

Conditional Operator

```
if (num % 2 == 0)
  System.out.println(num + "is even");
else
  System.out.println(num + "is odd");
```

```
System.out.println(
  (num % 2 == 0)? num + "is even" :
  num + "is odd");
```

Formatting Output

- Use the printf statement.

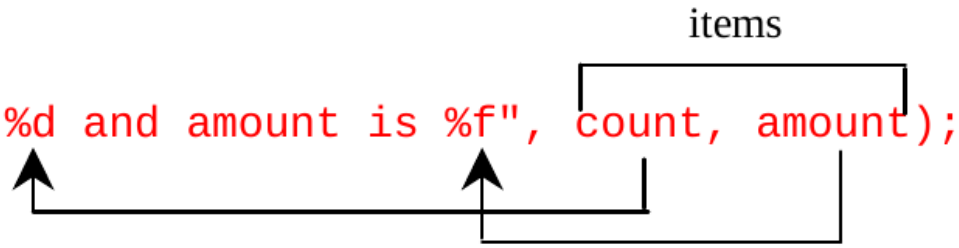
```
System.out.printf(format, items);
```

- Where format is a string that may consist of substrings and format specifiers. A format specifier specifies how an item should be displayed. An item may be a numeric value, character, boolean value, or a string. Each specifier begins with a percent sign.

Frequently-Used Specifiers

Specifier	Output	Example
%b	a boolean value	true or false
%c	a character	'a'
%d	a decimal integer	200
%f	a floating-point number	45.460000
%e	a number in standard scientific notation	4.556000e+01
%s	a string	"Java is cool"

```
int count = 5;
double amount = 45.56;
System.out.printf("count is %d and amount is %f", count, amount);
```




```
display          count is 5 and amount is 45.560000
```

Operator Precedence

- **`var++`**, **`var--`**
- **`+`**, **`-`** (Unary plus and minus), **`++var`**, **`--var`**
- **`(type)`** Casting
- **`!`** (Not)
- **`*`**, **`/`**, **`%`** (Multiplication, division, and remainder)
- **`+`**, **`-`** (Binary addition and subtraction)
- **`<`**, **`<=`**, **`>`**, **`>=`** (Comparison)
- **`==`**, **`!=`** (Equality)
- **`^`** (Exclusive OR)
- **`&&`** (Conditional AND) Short-circuit AND
- **`||`** (Conditional OR) Short-circuit OR
- **`=`**, **`+=`**, **`-=`**, **`*=`**, **`/=`**, **`%=`** (Assignment operator)

Operator Precedence and Associativity

The expression in the parentheses is evaluated first. (Parentheses can be nested, in which case the expression in the inner parentheses is executed first.) When evaluating an expression without parentheses, the operators are applied according to the precedence rule and the associativity rule.

If operators with the same precedence are next to each other, their associativity determines the order of evaluation. All binary operators except assignment operators are left-associative.

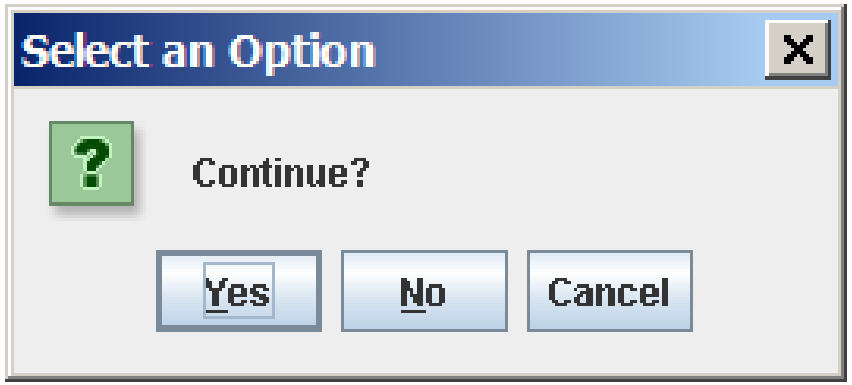
```
a - b + c - d is equivalent to ((a - b) + c) - d
```

Assignment operators are right-associative. Therefore, the expression

```
a = b += c = 5 is equivalent to a = (b += (c = 5))
```

(GUI) Confirmation Dialogs

```
int option = JOptionPane.showConfirmDialog
   (null, "Continue");
```



Java basics

Mathematical Functions, Characters, and Strings

Mathematical Functions and the `Math` Class

Java provides many useful methods in the **`Math`** class for performing common mathematical functions.

- Class constants:
  - **`PI`**
  - **`E`**
- Class methods:
  - Trigonometric Methods
  - Exponent Methods
  - Rounding Methods
  - **`min`**, **`max`**, **`abs`**, and **`random`** Methods

Trigonometric Methods

```
sin(double a)
cos(double a)
tan(double a)
acos(double a)
asin(double a)
atan(double a)
```

Radians

```
toRadians(90)
```

Examples:

```
Math.sin(0) returns 0.0
Math.sin(Math.PI / 6)
  returns 0.5
Math.sin(Math.PI / 2)
  returns 1.0
Math.cos(0) returns 1.0
Math.cos(Math.PI / 6)
  returns 0.866
Math.cos(Math.PI / 2)
  returns 0
```

Exponent Methods

`exp(double a)`

Returns e raised to the power of a.

`log(double a)`

Returns the natural logarithm of a.

`log10(double a)`

Returns the 10-based logarithm of a.

`pow(double a, double b)`

Returns a raised to the power of b.

`sqrt(double a)`

Returns the square root of a.

```
Examples:

Math.exp(1) returns 2.71
Math.log(2.71) returns 1.0
Math.pow(2, 3) returns 8.0
Math.pow(3, 2) returns 9.0
Math.pow(3.5, 2.5) returns
  22.91765
Math.sqrt(4) returns 2.0
Math.sqrt(10.5) returns 3.24
```

Rounding Methods

`double ceil(double x)`

x rounded up to its nearest integer. This integer is returned as a double value.

`double floor(double x)`

x is rounded down to its nearest integer. This integer is returned as a double value.

`double rint(double x)`

x is rounded to its nearest integer. If x is equally close to two integers, the even one is returned as a double.

`int round(float x)`

Return **`(int)Math.floor(x+0.5)`**.

`long round(double x)`

Return **`(long)Math.floor(x+0.5)`**.

Rounding Methods Examples

```
Math.ceil(2.1) returns 3.0
Math.ceil(2.0) returns 2.0
Math.ceil(-2.0) returns -2.0
Math.ceil(-2.1) returns -2.0
Math.floor(2.1) returns 2.0
Math.floor(2.0) returns 2.0
Math.floor(-2.0) returns -2.0
Math.floor(-2.1) returns -3.0
```

```
Math.rint(2.1) returns 2.0
Math.rint(2.0) returns 2.0
Math.rint(-2.0) returns -2.0
Math.rint(-2.1) returns -2.0
Math.rint(2.5) returns 2.0
Math.rint(-2.5) returns -2.0
Math.round(2.6f) returns 3
Math.round(2.0) returns 2
Math.round(-2.0f) returns -2
Math.round(-2.6) returns -3
```

`min`, `max`, and `abs`

`max(a, b)` and `min(a, b)`

Returns the maximum or minimum of two parameters.

`abs(a)`

Returns the absolute value of the parameter.

`random()`

Returns a random **`double`** value in the range [0.0, 1.0).

```
Examples:

Math.max(2, 3) returns 3
Math.max(2.5, 3) returns
  3.0
Math.min(2.5, 3.6)
  returns 2.5
Math.abs(-2) returns 2
Math.abs(-2.1) returns
  2.1
```

The `random` Method

Generates a random **`double`** value greater than or equal to 0.0 and less than 1.0 (**`0 <= Math.random() < 1.0`**).

Examples:

```
(int)(Math.random() * 10)  ──────►  Returns a random integer
                                    between 0 and 9.

50 + (int)(Math.random() * 50) ──►  Returns a random integer
                                    between 50 and 99.
```

In general,

```
a + Math.random() * b  ──────►  Returns a random number between
                                a and a + b, excluding a + b.
```

Character Data Type

```
char letter = 'A'; (ASCII)
char numChar = '4'; (ASCII)
char letter = '\u0041'; (Unicode)
char numChar = '\u0034'; (Unicode)
```

Four hexadecimal digits.

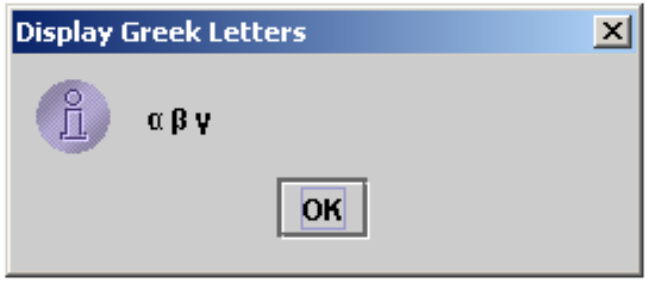
NOTE: The increment and decrement operators can also be used on **`char`** variables to get the next or preceding Unicode character. For example, the following statements display character b.

```
char ch = 'a';
System.out.println(++ch);
```

Unicode Format

Java characters use Unicode, a 16-bit encoding scheme established by the Unicode Consortium to support the interchange, processing, and display of written texts in the world's diverse languages. Unicode takes two bytes, preceded by \u, expressed in four hexadecimal numbers that run from **`'\u0000'`** to **`'\uFFFF'`**. So, Unicode can represent 65535 + 1 characters.

```
Unicode \u03b1 \u03b2 \u03b3 for three Greek letters
```



Escape Sequences for Special Characters

Description	Escape Sequence	Unicode
Backspace	`\b`	`\u0008`
Tab	`\t`	`\u0009`
Linefeed	`\n`	`\u000A`
Carriage return	`\r`	`\u000D`
Backslash	`\\`	`\u005C`
Single Quote	`\'`	`\u0027`
Double Quote	`\"`	`\u0022`

Casting between `char` and Numeric Types

```
int i = 'a'; // Same as int i = (int)'a';

char c = 97; // Same as char c = (char)97;
```

Comparing and Testing Characters

```
if (ch >= 'A' && ch <= 'Z')
  System.out.println(ch + " is an uppercase letter");
else if (ch >= 'a' && ch <= 'z')
  System.out.println(ch + " is a lowercase letter");
else if (ch >= '0' && ch <= '9')
  System.out.println(ch + " is a numeric character");
```

Methods in the `Character` Class

Method	Description
`isDigit(ch)`	Returns true if the specified character is a digit.
`isLetter(ch)`	Returns true if the specified character is a letter.
`isLetterOfDigit(ch)`	Returns true if the specified character is a letter or digit.
`isLowerCase(ch)`	Returns true if the specified character is a lowercase letter.
`isUpperCase(ch)`	Returns true if the specified character is an uppercase letter.
`toLowerCase(ch)`	Returns the lowercase of the specified character.
`toUpperCase(ch)`	Returns the uppercase of the specified character.

The `String` Type

The **`char`** type only represents one character. To represent a string of characters, use the data type called **`String`**. For example,

```
String message = "Welcome to Java";
```

`String` is actually **a predefined class** in the Java library just like **the `System` class** and **`JOptionPane` class**. The **`String`** type is not a primitive type.

Simple Methods for `String` Objects

`Method`	**Description**
`length()`	Returns the number of characters in this string.
`charAt(index)`	Returns the character at the specified index from this string.
`concat(s1)`	Returns a new string that concatenates this string with string s1.
`toUpperCase()`	Returns a new string with all letters in uppercase.
`toLowerCase()`	Returns a new string with all letters in lowercase.
`trim()`	Returns a new string with whitespace characters trimmed on both sides.

Getting String Length

```
String message = "Welcome to Java";
System.out.println("The length of " + message + " is "
  + message.length());
```

String Concatenation

```
String s3 = s1.concat(s2); or String s3 = s1 + s2;

// Three strings are concatenated
String message = "Welcome " + "to " + "Java";

// String Chapter is concatenated with number 2
String s = "Chapter" + 2; // s becomes Chapter2

// String Supplement is concatenated with character B
String s1 = "Supplement" + 'B'; // s1 becomes SupplementB
```

Comparing Strings

Method	Description
`equals(s1)`	Returns true if this string is equal to string s1.
`equalsIgnoreCase(s1)`	Returns true if this string is equal to string s1; it is case insensitive.
`compareTo(s1)`	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
`compareToIgnoreCase(s1)`	Same as compareTo except that the comparison is case insensitive.
`startsWith(prefix)`	Returns true if this string starts with the specified prefix.
`endsWith(suffix)`	Returns true if this string ends with the specified suffix.
`==`	Never use == to compare strings: it compares references, not content. Use equals(s1).

▸ OrderTwoCities

Java basics

Loops

Problem: Printing “Welcome to Java” 100 times

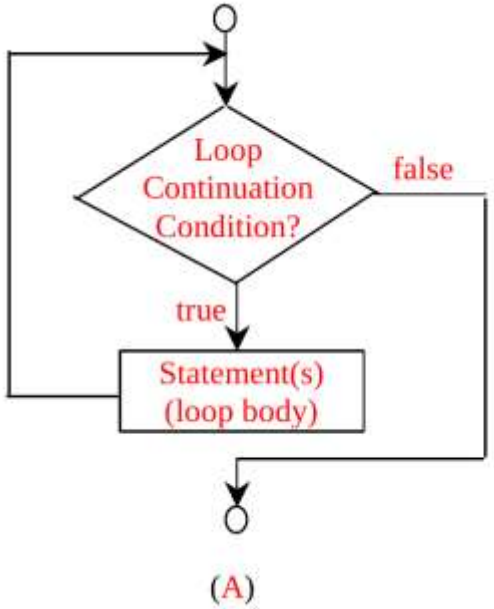
100 times

```
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
...
...
...
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
```

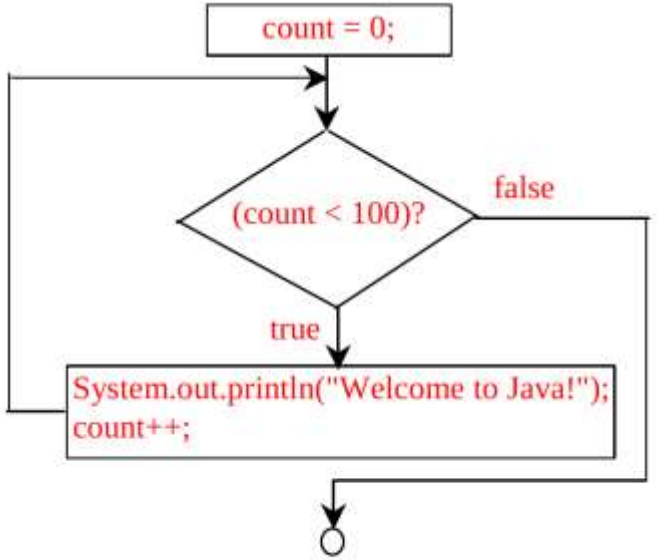
`while` Loop

```
while (loop-continuation-condition) {
  // loop-body;
  Statement(s);
}
```

```
int count = 0;
while (count < 100) {
  System.out.println("Welcome to Java!");
  count++;
}
```

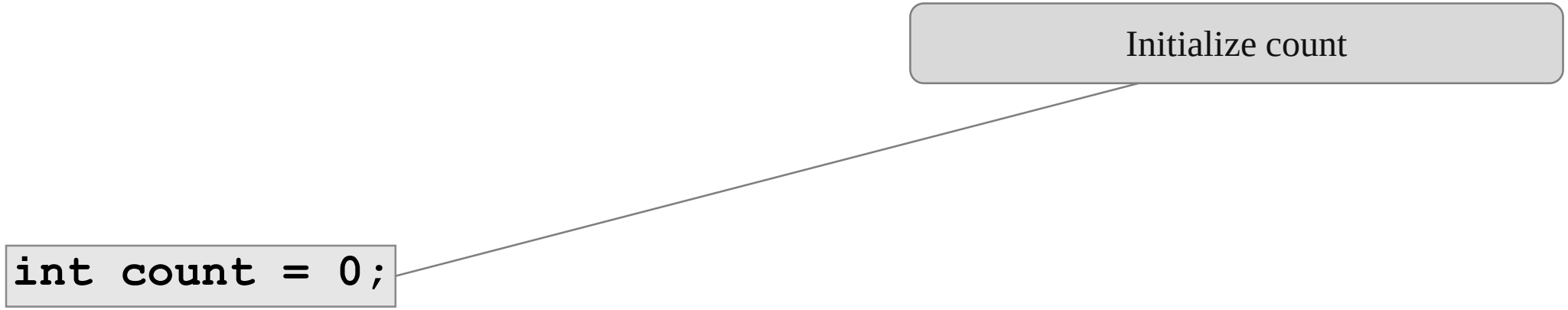


(A)



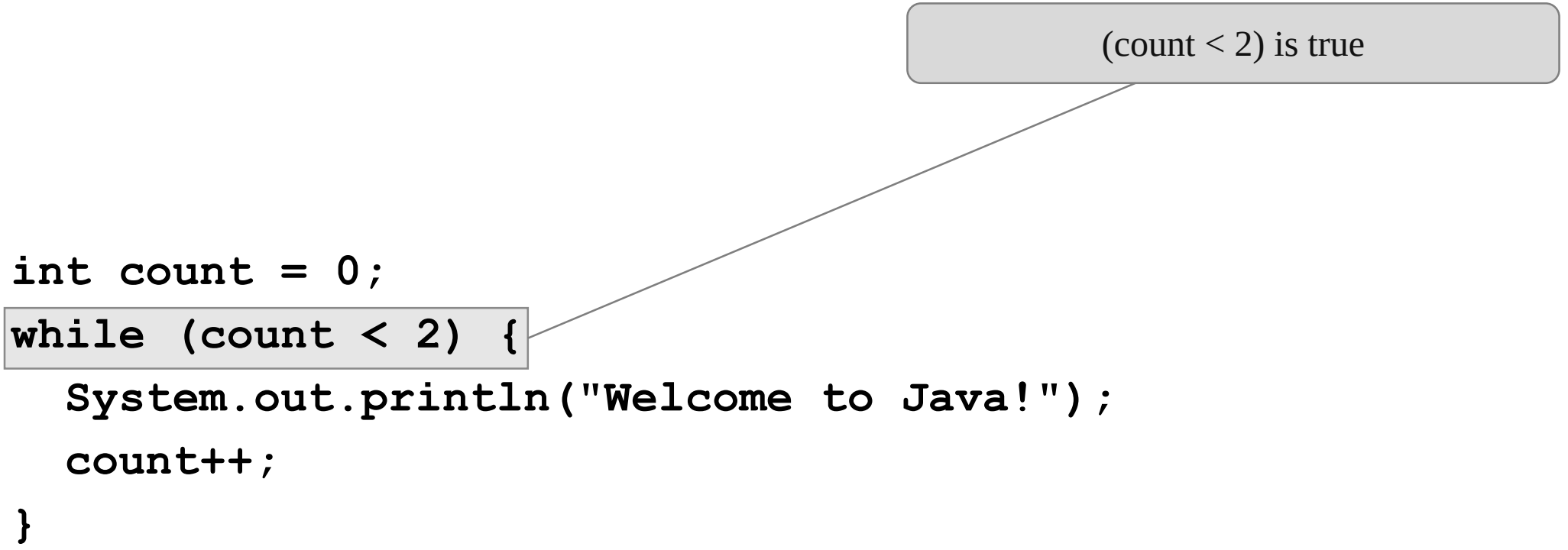
(B)

Trace `while` Loop

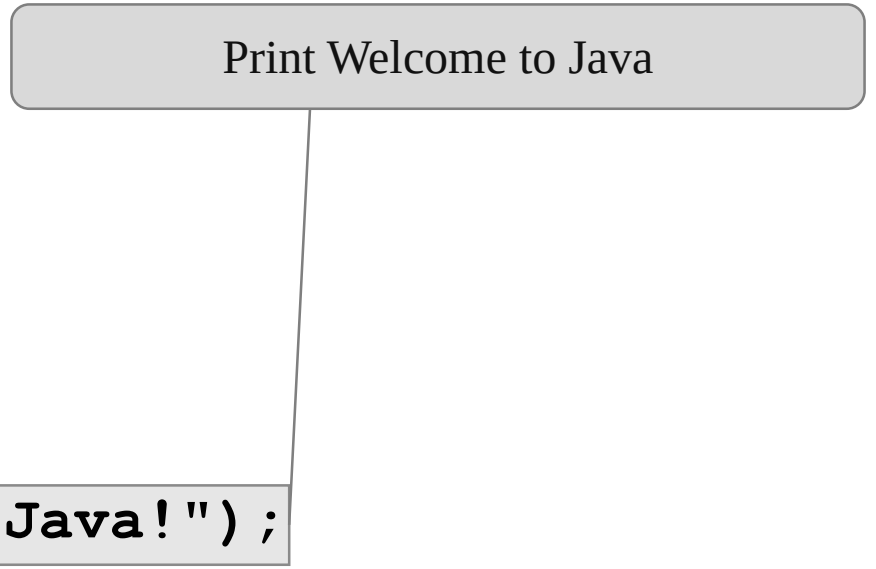



```
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Trace `while` Loop

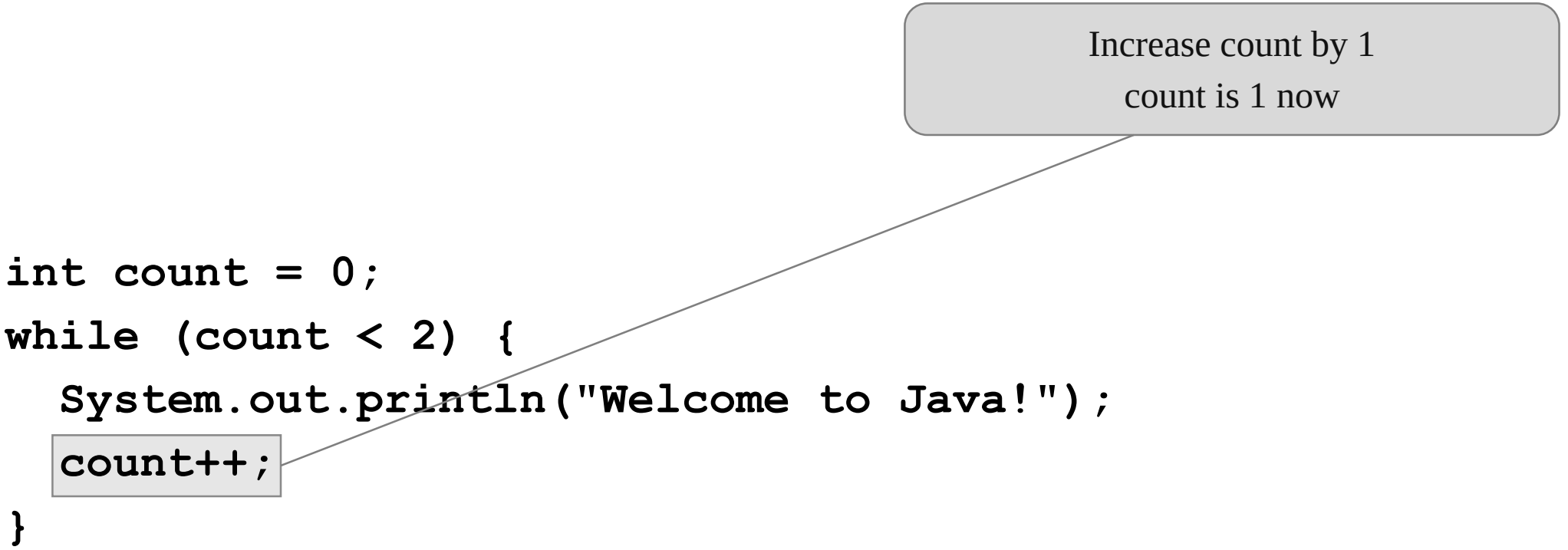


Trace `while` Loop




```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Trace `while` Loop

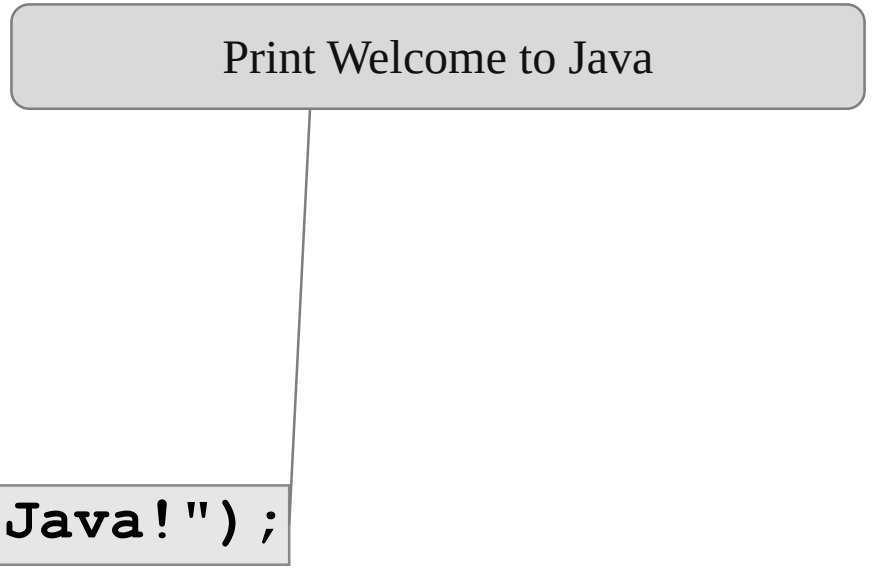


Trace `while` Loop

(count < 2) is still true since count is 1

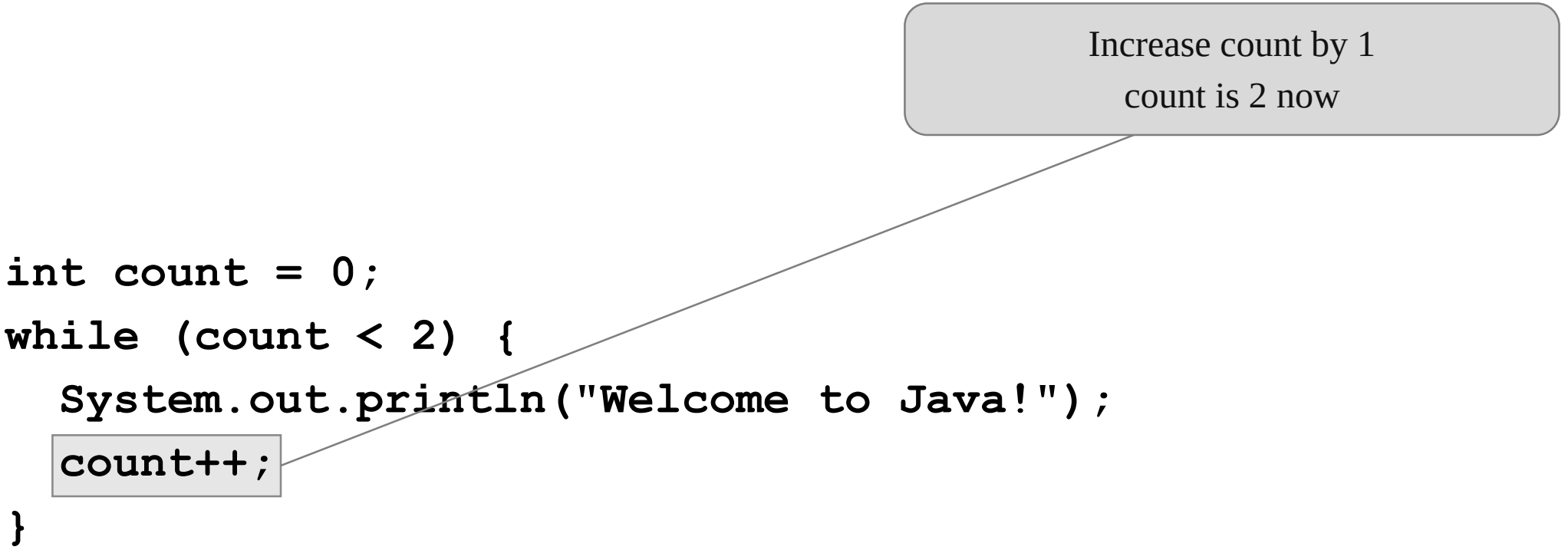
```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Trace `while` Loop




```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Trace `while` Loop



Trace `while` Loop

(count < 2) is false since count is 2 now

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Trace `while` Loop

The loop exits. Execute the next statement after the loop.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Ending a Loop with a Sentinel Value

Often the number of times a loop is executed is not predetermined. You may use an input value to signify the end of the loop. Such a value is known as a sentinel value.

Write a program that reads and calculates the sum of an unspecified number of integers. The input 0 signifies the end of the input.

▶ SentinelValue

Caution

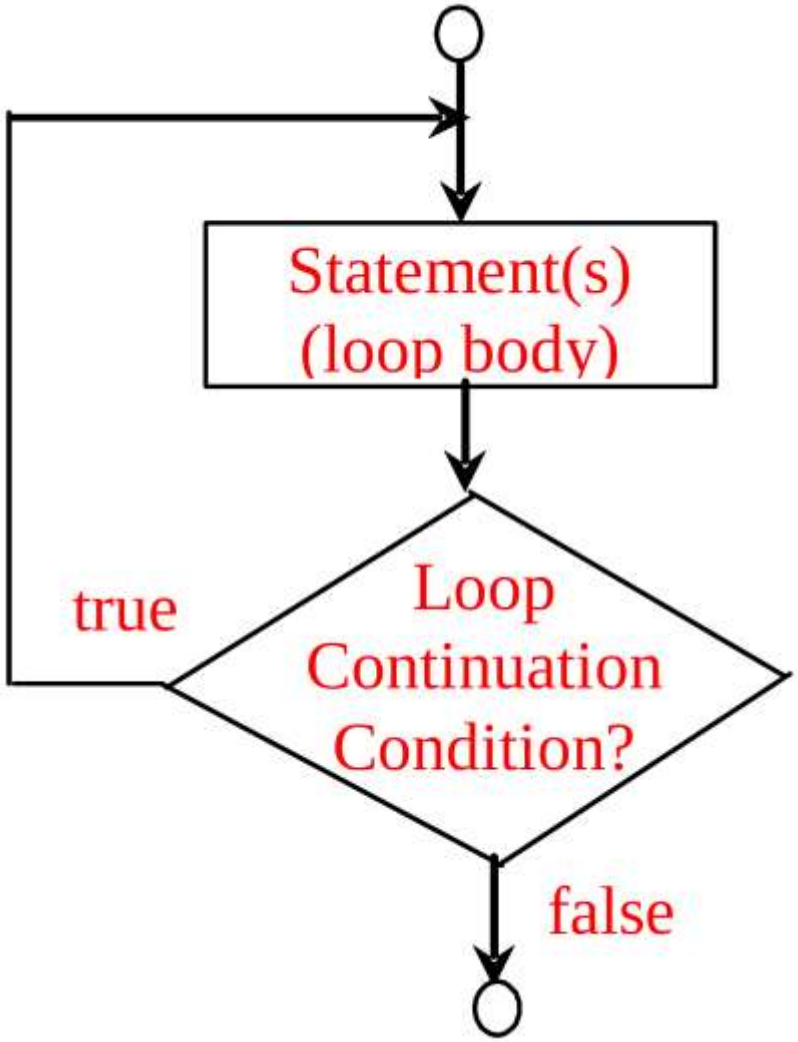
Don’t use floating-point values for equality checking in a loop control. Since floating-point values are approximations for some values, using them could result in imprecise counter values and inaccurate results. Consider the following code for computing 1 + 0.9 + 0.8 + ... + 0.1:

```
double item = 1; double sum = 0;
while (item != 0) { // No guarantee item will be 0
  sum += item;
  item -= 0.1;
}
System.out.println(sum);
```

Variable **`item`** starts with 1 and is reduced by 0.1 every time the loop body is executed. The loop should terminate when **`item`** becomes 0. However, there is no guarantee that **`item`** will be exactly 0, because the floating-point arithmetic is approximated. This loop seems OK on the surface, but it is actually an infinite loop.

do-while Loop

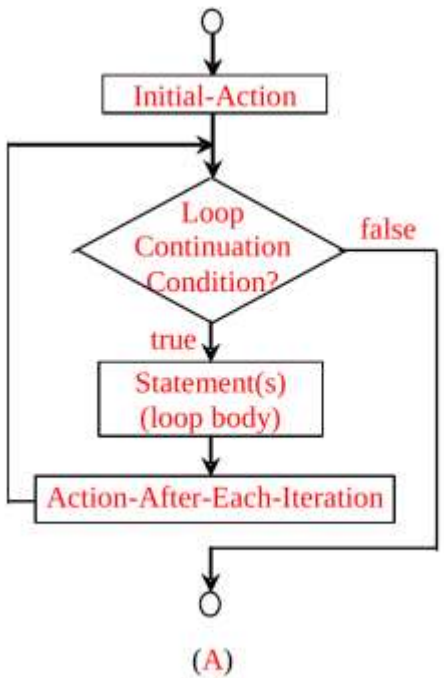
```
do {
  // Loop body;
  Statement(s);
} while (loop-continuation-condition);
```



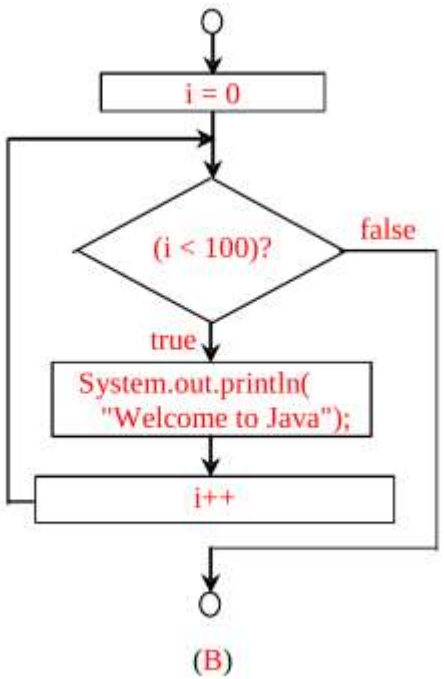
`for` Loops

```
for (initial-action; loop-continuation-condition;
     action-after-each-iteration) {
  // loop body;
  Statement(s);
}
```

```
int i;
for (i = 0; i < 100; i++) {
  System.out.println(
    "Welcome to Java!");
}
```

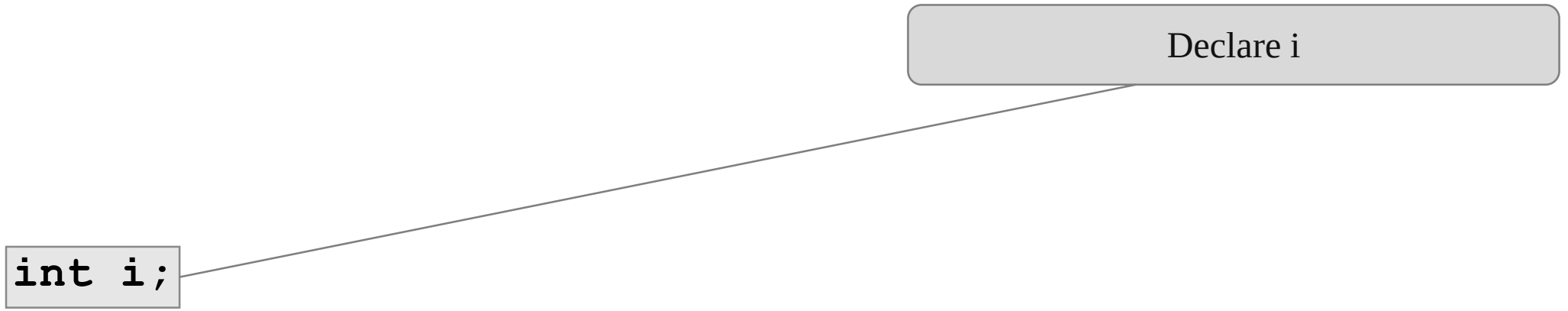


(A)



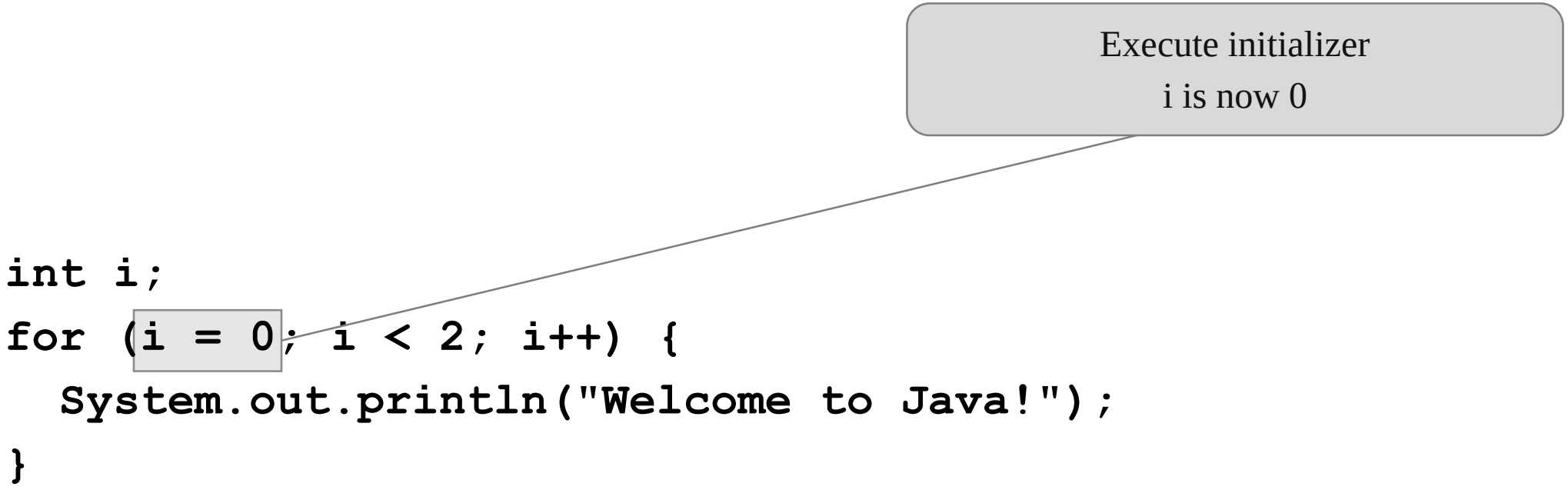
(B)

Trace `for` Loop




```
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Trace `for` Loop



Trace `for` Loop

(i < 2) is true
since i is 0

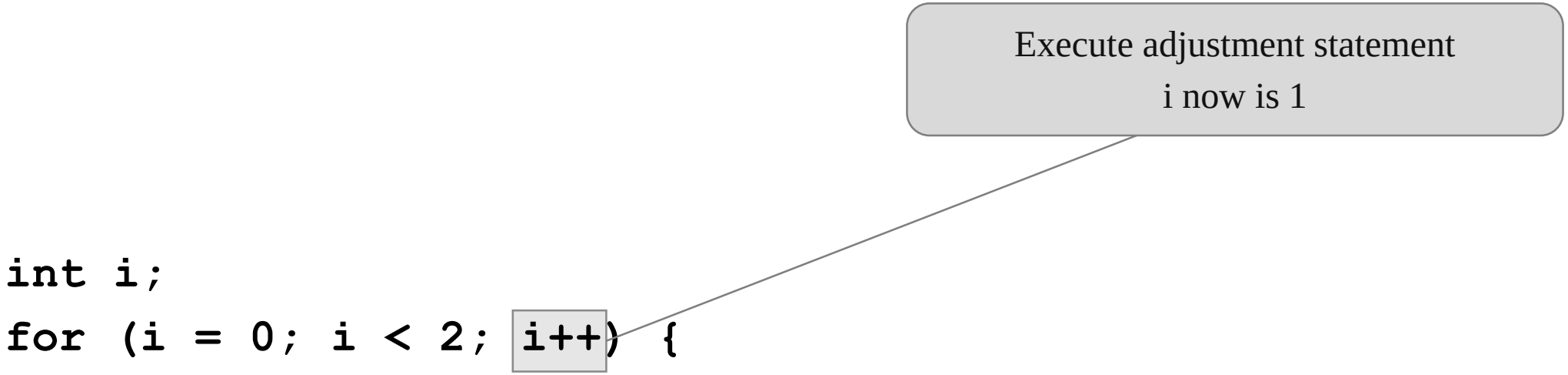
```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Trace `for` Loop

Print Welcome to Java

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Trace `for` Loop



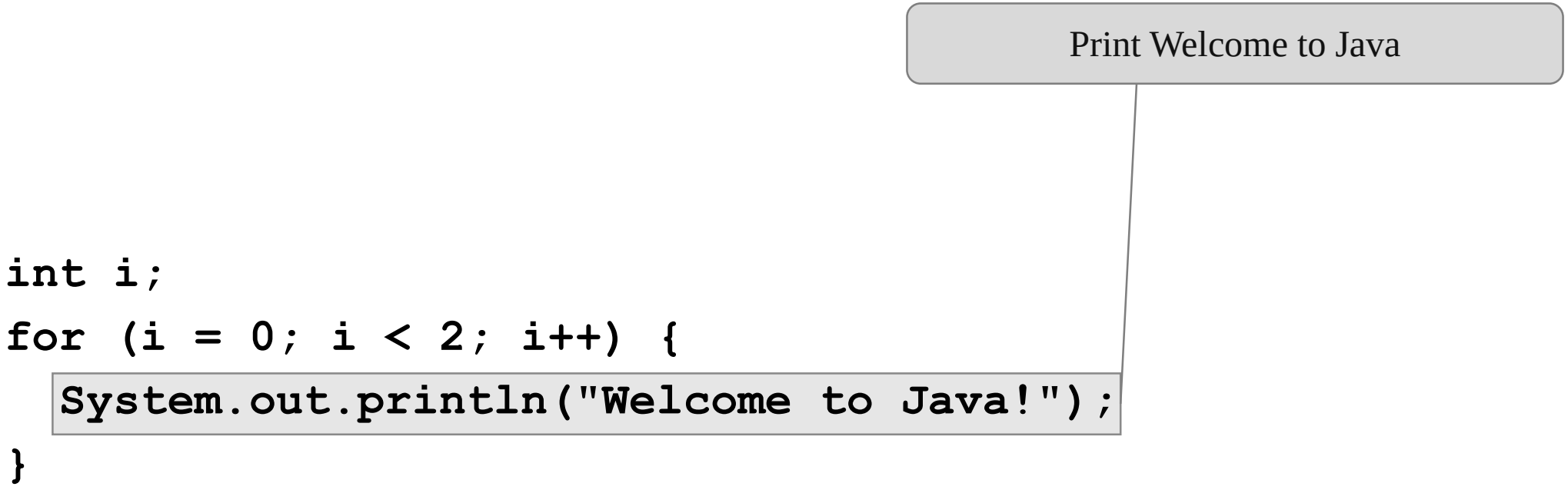

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Trace `for` Loop

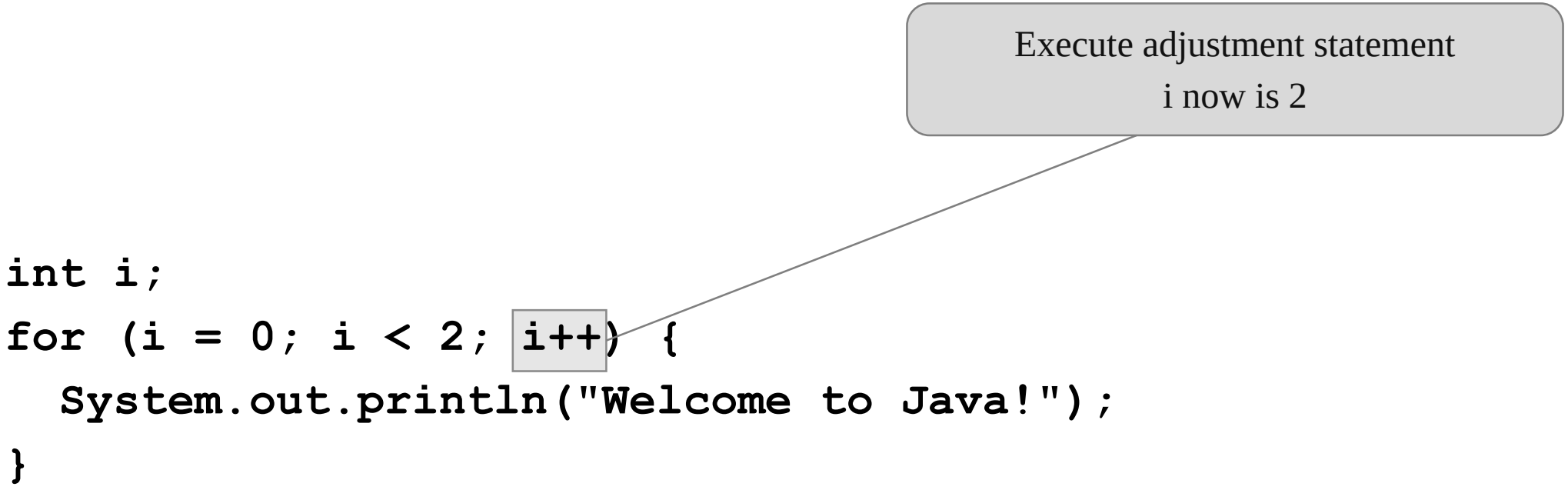
(i < 2) is still true
since i is 1

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Trace `for` Loop



Trace `for` Loop



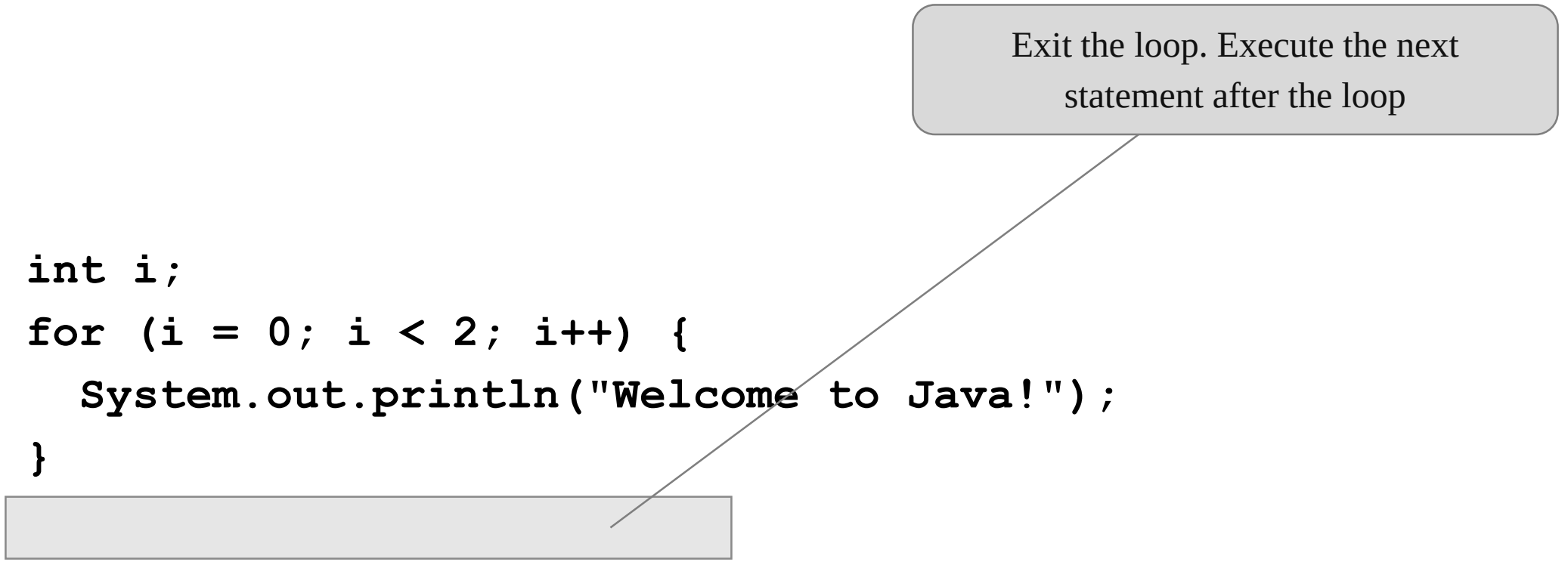
Trace `for` Loop

(i < 2) is false
since i is 2

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Trace `for` Loop

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```



Note

The initial-action in a **`for`** loop can be a list of zero or more comma-separated expressions. The action-after-each-iteration in a **`for`** loop can be a list of zero or more comma-separated statements. Therefore, the following two **`for`** loops are correct. They are rarely used in practice, however.

```
for (int i = 1; i < 100; System.out.println(i++));

for (int i = 0, j = 0; (i + j < 10); i++, j++) {
  // Do something
}
```

Note

If the loop-continuation-condition in a **`for`** loop is omitted, it is implicitly true.

Thus the statement given below in (a), which is an infinite loop, is correct.

Nevertheless, it is better to use the equivalent loop in (b) to avoid confusion:

```
for ( ; ; ) {
  // Do something
}
```

(a)

Equivalent

```
while (true) {
  // Do something
}
```

(b)

Caution

Adding a semicolon at the end of the **`for`** clause before the loop body is a common mistake, as shown below:

Logic Error

```
for (int i=0; i<10; i++);
{
  System.out.println("i is " + i);
}
```

Similarly, the following loop is also wrong:

```
int i=0;
while (i < 10);
{
  System.out.println("i is " + i);
  i++;
}
```

Caution, cont.

In the case of the **`do`** loop, the following semicolon is needed to end the loop.

```
int i=0;
do {
  System.out.println("i is " + i);
  i++;
} while (i<10);
```

Which Loop to Use?

The three forms of loop statements, **`while`**, **`do-while`**, and **`for`**, are expressively equivalent.
For example, a **`while`** loop in (a) in the following figure can always be converted into the following **`for`** loop in (b):

```
while (loop-continuation-condition) {
  // Loop body
}
```

(a)

Equivalent

```
for ( ; loop-continuation-condition; )
  // Loop body
}
```

(b)

- A **`for`** loop in (a) in the following figure can generally be converted into the following **`while`** loop in (b) except in certain special cases

```
for (initial-action;
     loop-continuation-condition;
     action-after-each-iteration) {
  // Loop body;
}
```

(a)

Equivalent

```
initial-action;
while (loop-continuation-condition) {
  // Loop body;
  action-after-each-iteration;
}
```

(b)

Recommendations

Use the one that is most intuitive and comfortable for you. In general:

- **`for`** loop may be used if the number of repetitions is known, as, for example, when you need to print a message 100 times

- **`while`** loop may be used if the number of repetitions is not known, as in the case of reading the numbers until the input is 0

- **`do-while`** loop can be used to replace a **`while`** loop if the loop body has to be executed before testing the continuation condition

Using `break` and `continue`

Examples for using the **`break`** and **`continue`** keywords:

```
public class TestBreak {
  public static void main(String[] args) {
    int sum = 0;
    int number = 0;

    while (number < 20) {
      number++;
      sum += number;
      if (sum >= 100)
        break;
    }

    System.out.println("The number is " + number);
    System.out.println("The sum is " + sum);
  }
}
```

```
public class TestContinue {
  public static void main(String[] args) {
    int sum = 0;
    int number = 0;

    while (number < 20) {
      number++;
      if (number == 10 || number == 11)
        continue;
      sum += number;
    }

    System.out.println("The sum is " + sum);
  }
}
```

▸ TestBreak ▸ TestContinue

(GUI) Controlling a Loop with a Confirmation Dialog

A sentinel-controlled loop can be implemented using a confirmation dialog. The answers Yes or No to continue or terminate the loop. The template of the loop may look as follows:

```
int option = 0;
while (option == JOptionPane.YES_OPTION) {
  System.out.println("continue loop");
  option = JOptionPane.showConfirmDialog(null, "Continue?");
}
```

Java basics

Methods

Problem

Find the sum of integers from 1 to 10, from 20 to 30, and from 35 to 45, respectively.

```
int sum = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
  sum += i;
System.out.println("Sum from 20 to 30 is " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
  sum += i;
System.out.println("Sum from 35 to 45 is " + sum);
```

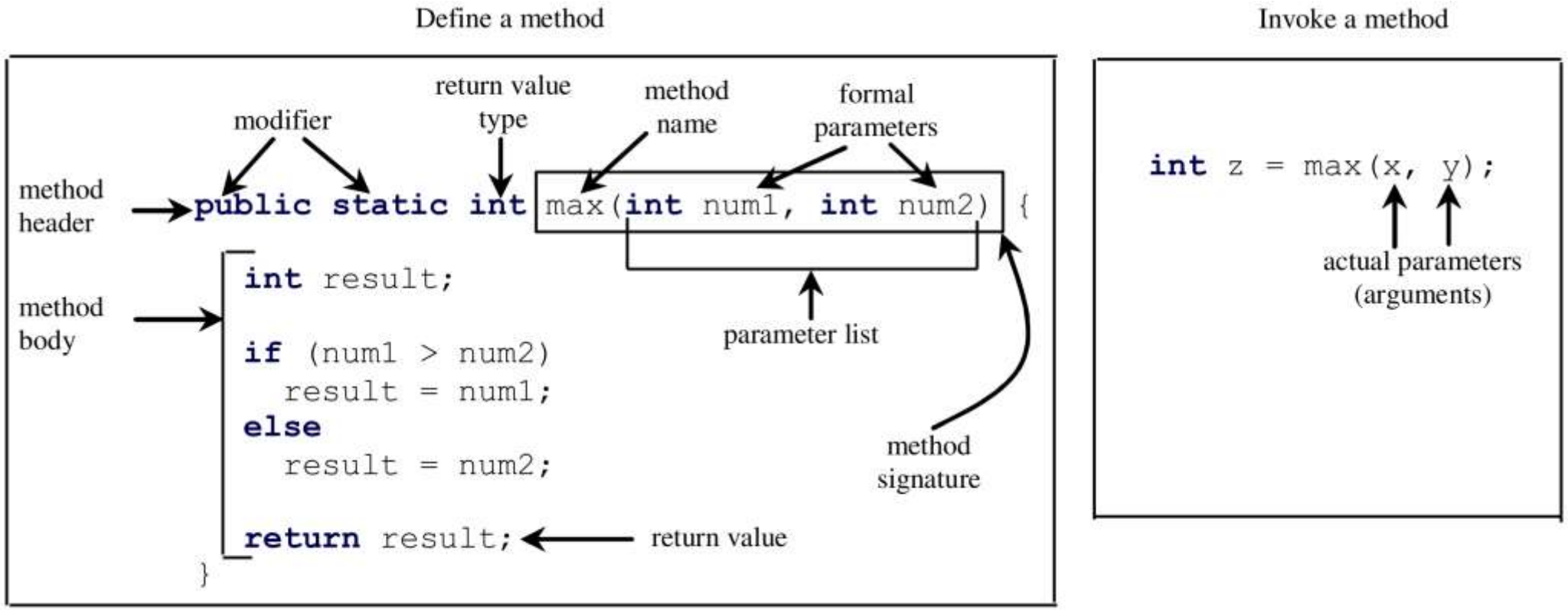
Solution

```
public static int sum(int i1, int i2) {
  int sum = 0;
  for (int i = i1; i <= i2; i++)
    sum += i;
  return sum;
}
```

```
public static void main(String[] args) {
  System.out.println("Sum from 1 to 10 is " + sum(1, 10));
  System.out.println("Sum from 20 to 30 is " + sum(20, 30));
  System.out.println("Sum from 35 to 45 is " + sum(35, 45));
}
```

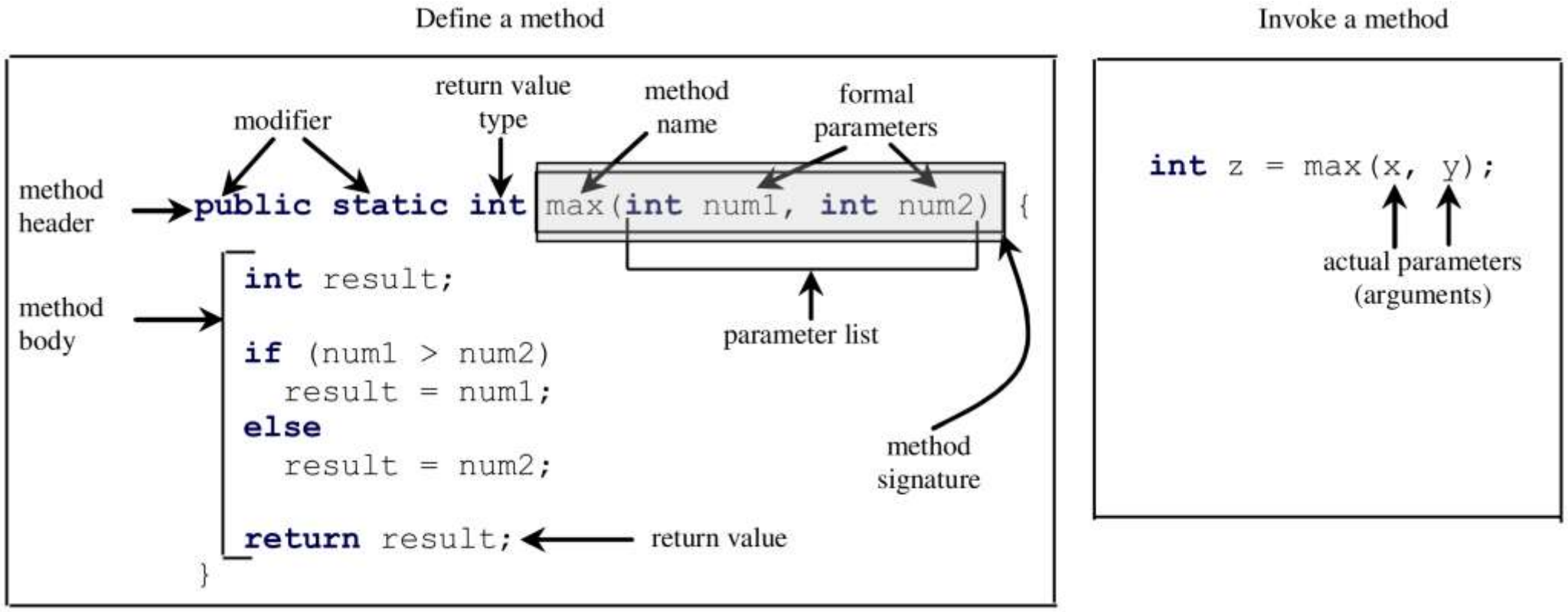
Defining Methods

A method is a collection of statements that are grouped together to perform an operation.



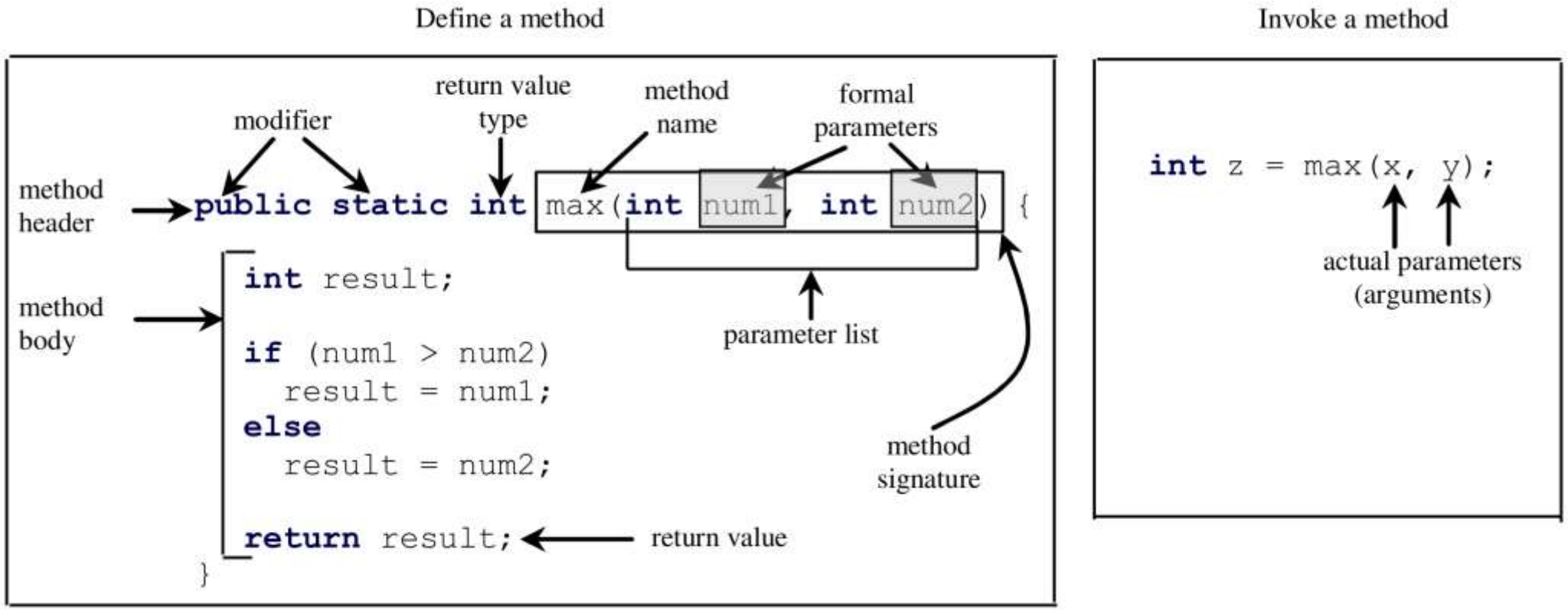
Method Signature

Method signature is the combination of the method name and the parameter list.



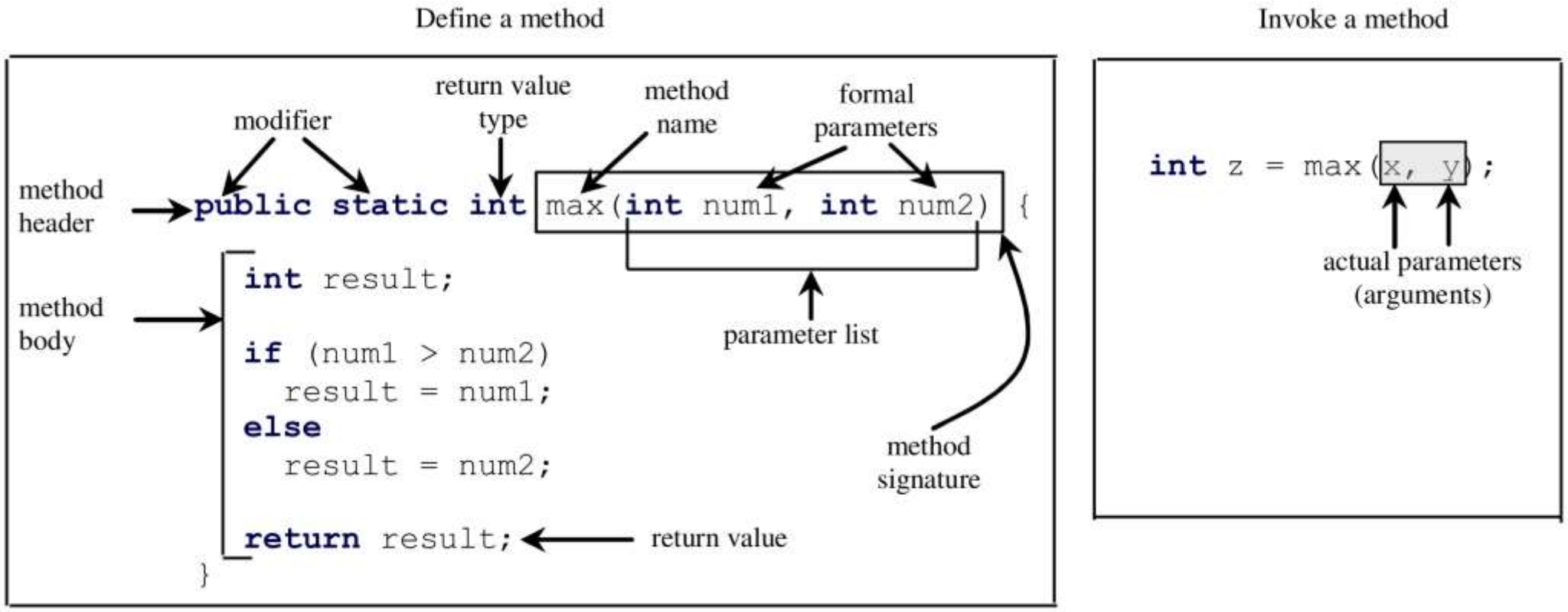
Formal Parameters

The variables defined in the method header are known as formal parameters.



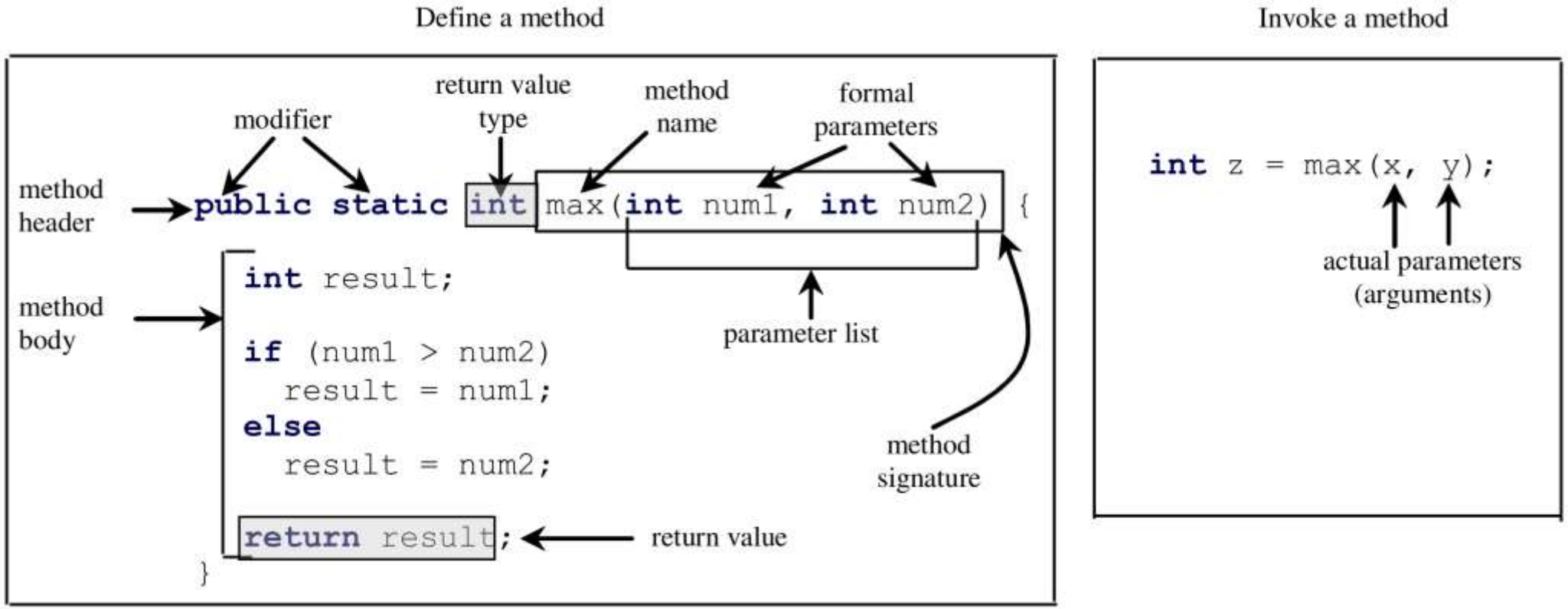
Actual Parameters

When a method is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument.



Return Value Type

A method may return a value. The returnValueType is the data type of the value the method returns. If the method does not return a value, the returnValueType is the keyword **`void`**. For example, the returnValueType in the **`main`** method is **`void`**.



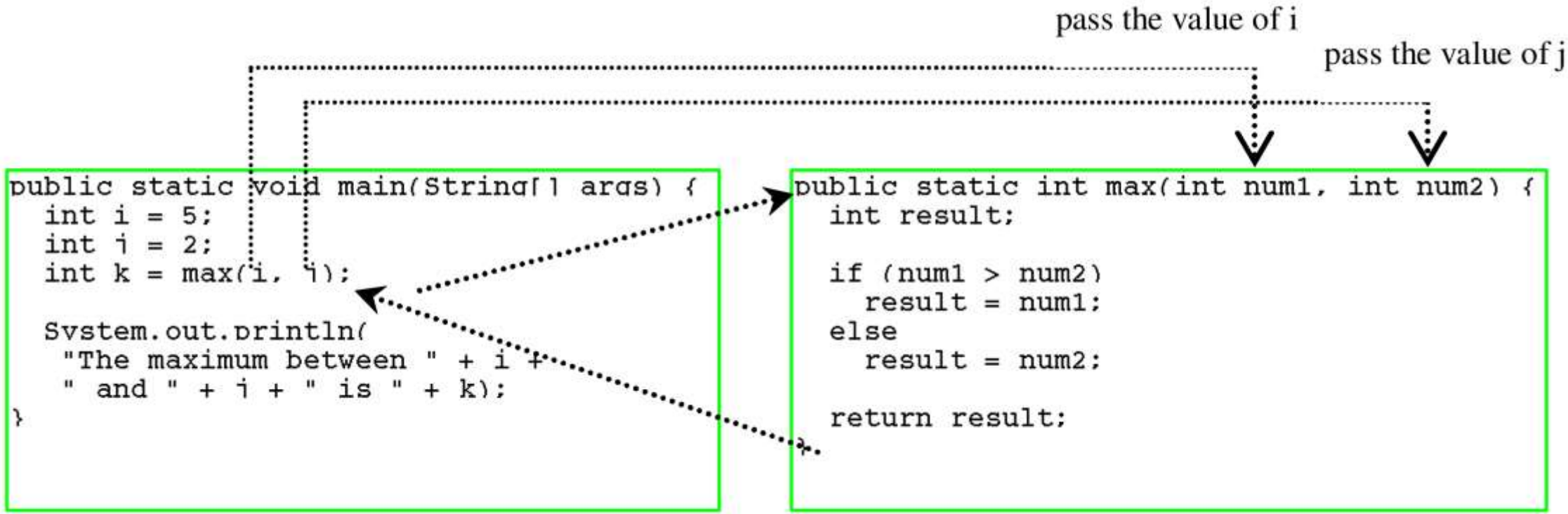
Calling Methods

Testing the **`max`** method

This program demonstrates calling a method **`max`** to return the largest of the **`int`** values

▸ TestMax

Calling Methods, cont.



Trace Method Invocation

i is now 5

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

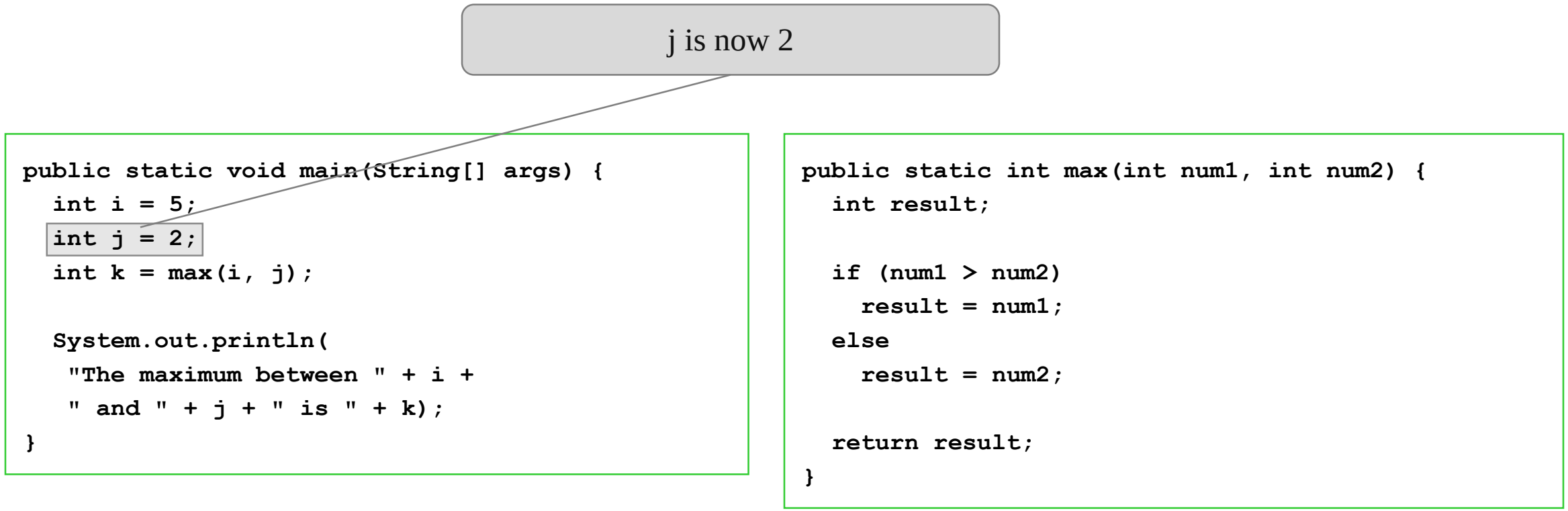
  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

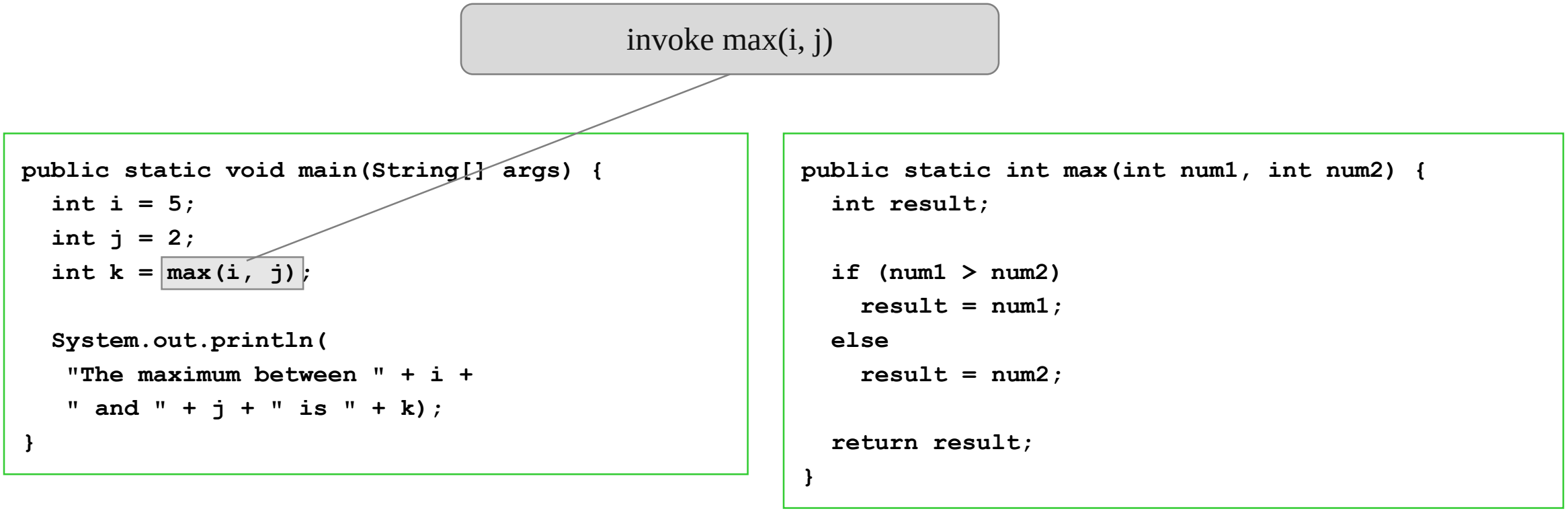
  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Trace Method Invocation



Trace Method Invocation



Trace Method Invocation

invoke max(i, j)
Pass the value of i to num1
Pass the value of j to num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Trace Method Invocation

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```




```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Trace Method Invocation

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

(num1 > num2) is true since num1 is 5 and num2 is 2

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Trace Method Invocation

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

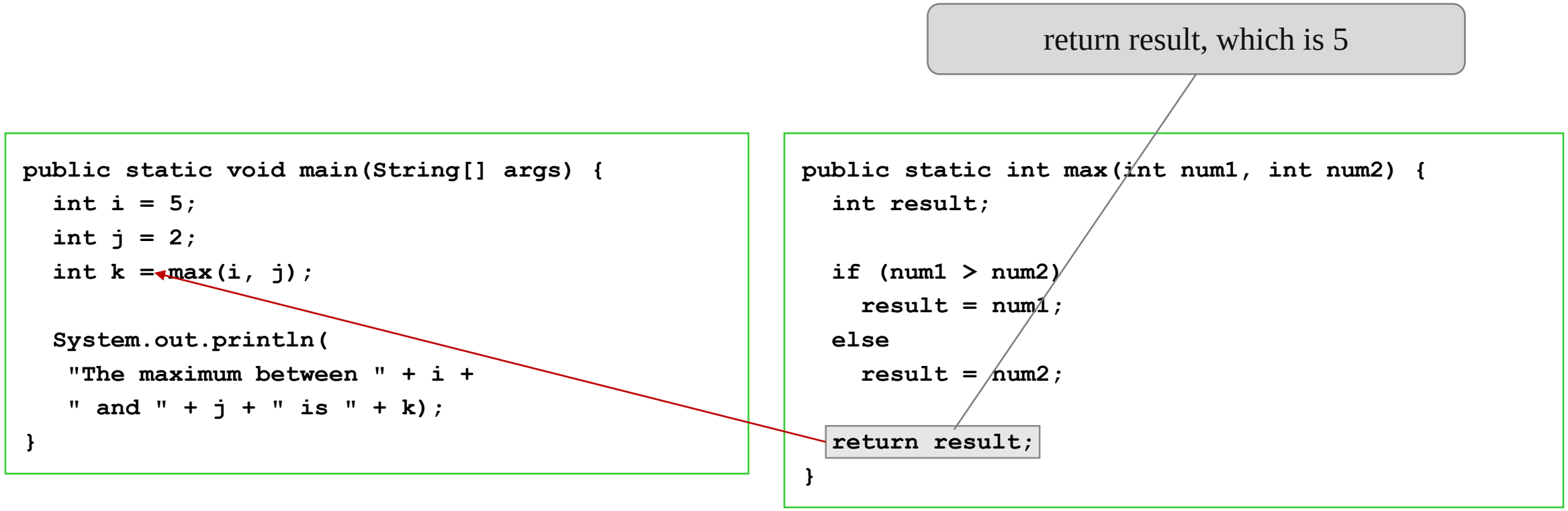
result is now 5

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Trace Method Invocation



Trace Method Invocation

return max(i, j) and assign the return value to k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Trace Method Invocation

Execute the print statement

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
    "The maximum between " + i +
    " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

CAUTION

A **`return`** statement is required for a value-returning method. The method shown below in (a) is logically correct, but it has a compilation error because the Java compiler thinks it possible that this method does not return any value.

```
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else if (n < 0)
    return -1;
}
```

(a)

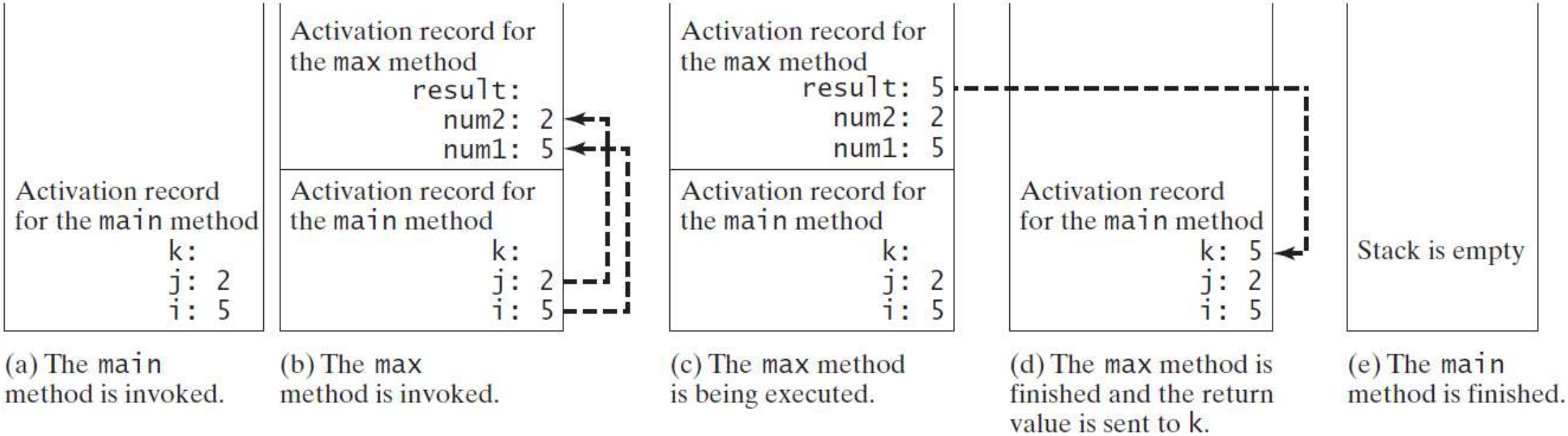
Should be

```
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else
    return -1;
}
```

(b)

To fix this problem, delete **`if (n < 0)`** in (a), so that the compiler will see a **`return`** statement to be reached regardless of how the **`if`** statement is evaluated.

Call Stacks



(a) The `main` method is invoked.

(b) The `max` method is invoked.

(c) The `max` method is being executed.

(d) The `max` method is finished and the return value is sent to `k`.

(e) The `main` method is finished.

Trace Call Stack

i is declared and initialized

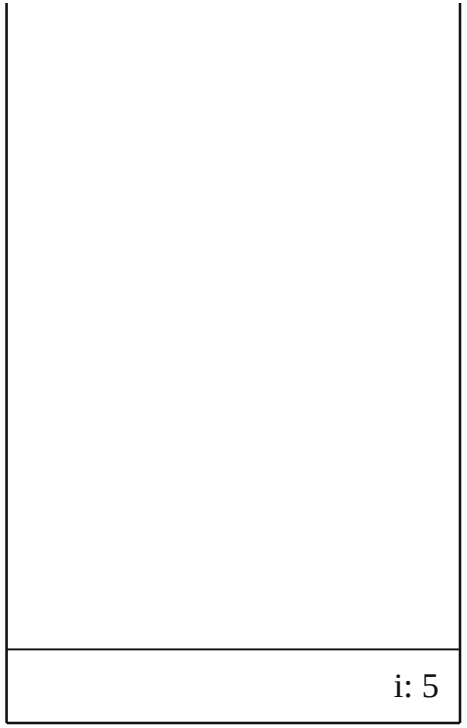
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The main method is invoked.

Trace Call Stack

j is declared and initialized

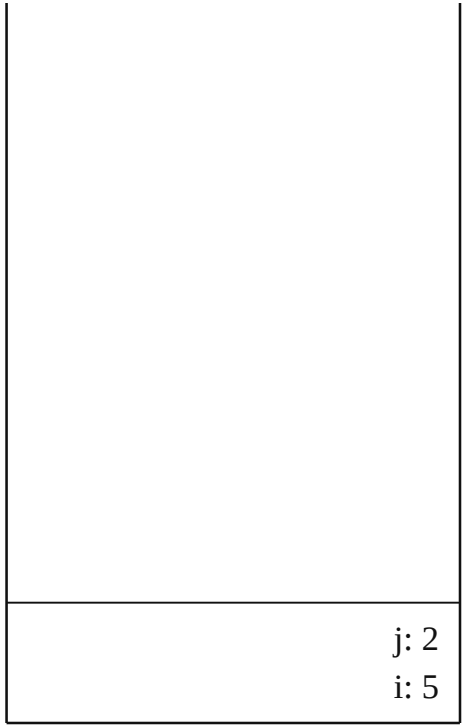
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The main method is invoked.

Trace Call Stack

Declare k

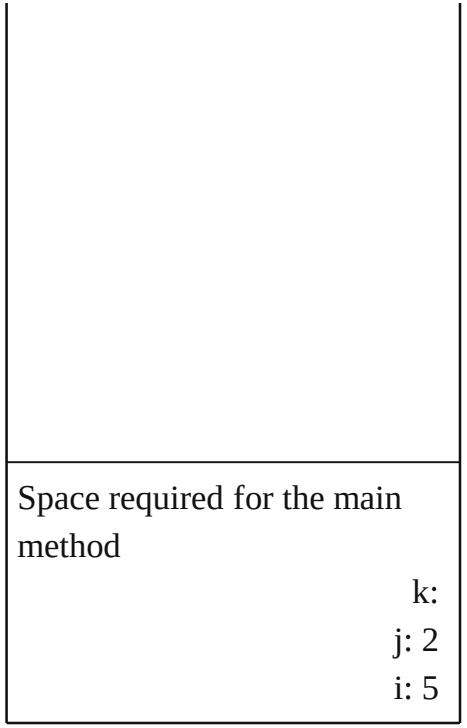
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The main method is invoked.

Trace Call Stack

Invoke max(i, j)

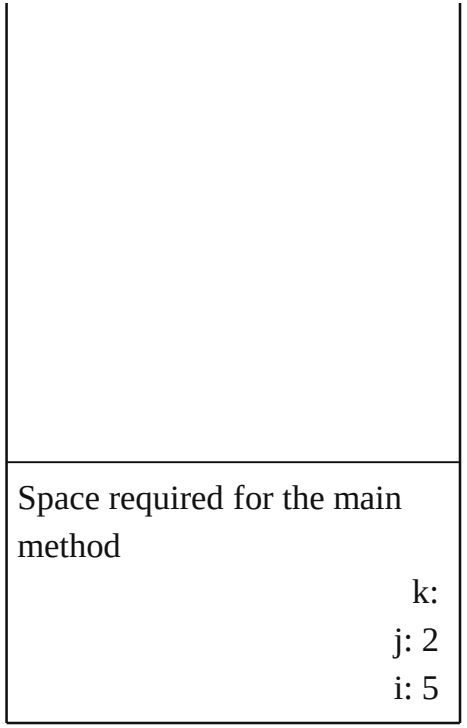
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The main method is invoked.

Trace Call Stack

pass the values of i and j to num1 and num2

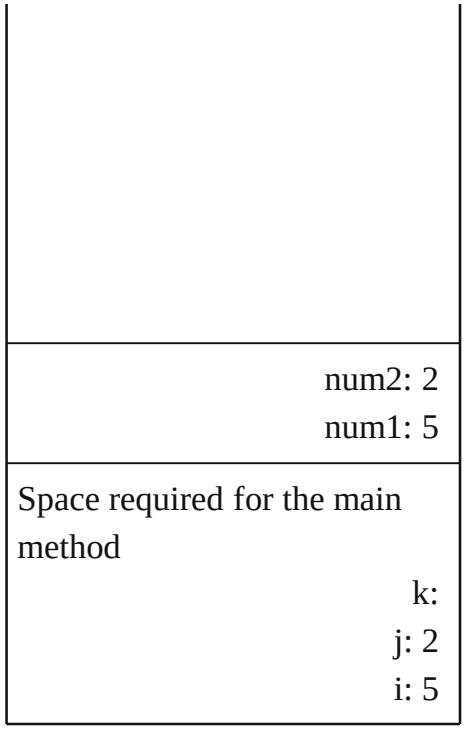
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

Trace Call Stack

Declare result

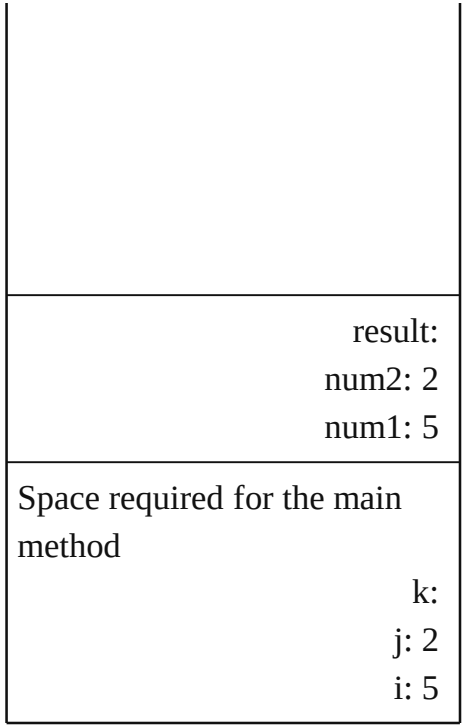
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

Trace Call Stack

(num1 > num2) is true

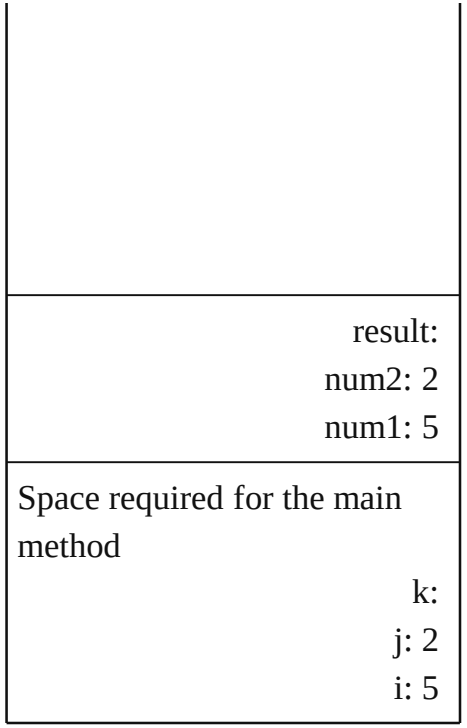
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

Trace Call Stack

Assign num1 to result

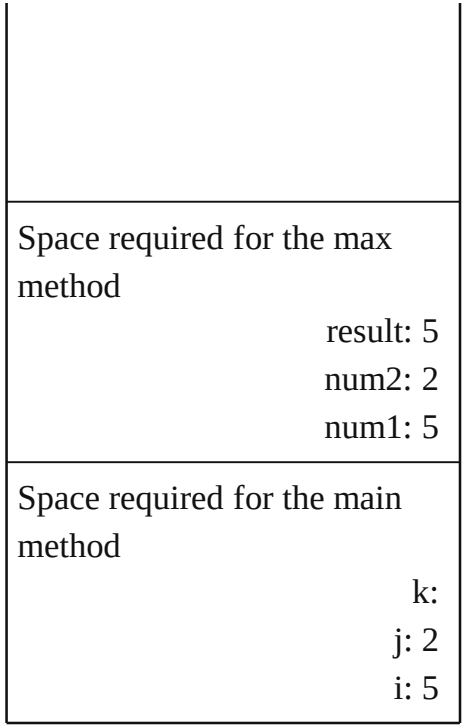
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

Trace Call Stack

Return result and assign it to k

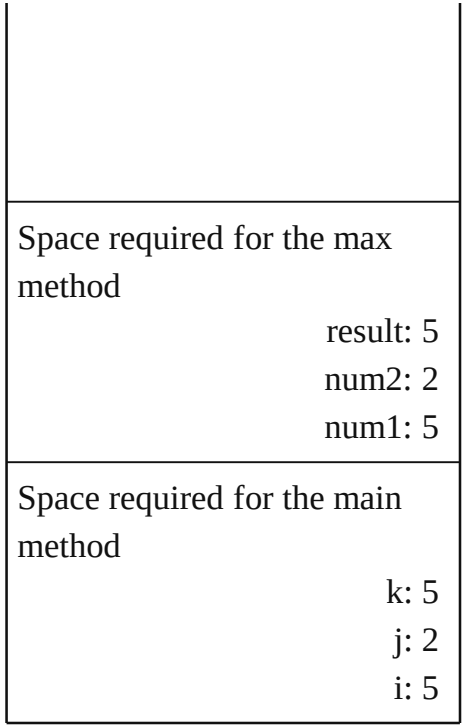
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

Trace Call Stack

Execute print statement

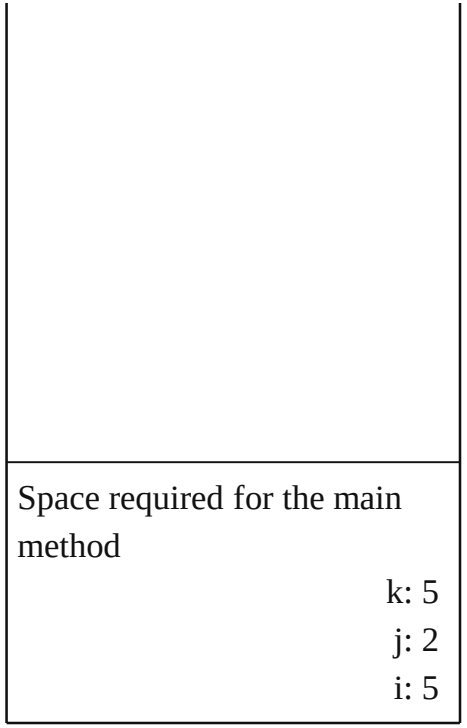
```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
    "The maximum between " + i +
    " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The main method is invoked.

`void` Method Example

This type of method does not return a value. The method performs some actions.

▸ TestVoidMethod ▸ TestReturnGradeMethod

Passing Parameters

```
public static void nPrintln(String message, int n) {
  for (int i = 0; i < n; i++)
    System.out.println(message);
}
```

Suppose you invoke the method using

```
nPrintln("Welcome to Java", 5);
```

What is the output?

Suppose you invoke the method using

```
nPrintln("Computer Science", 15);
```

What is the output?

Can you invoke the method using

```
nPrintln(15, "Computer Science");
```

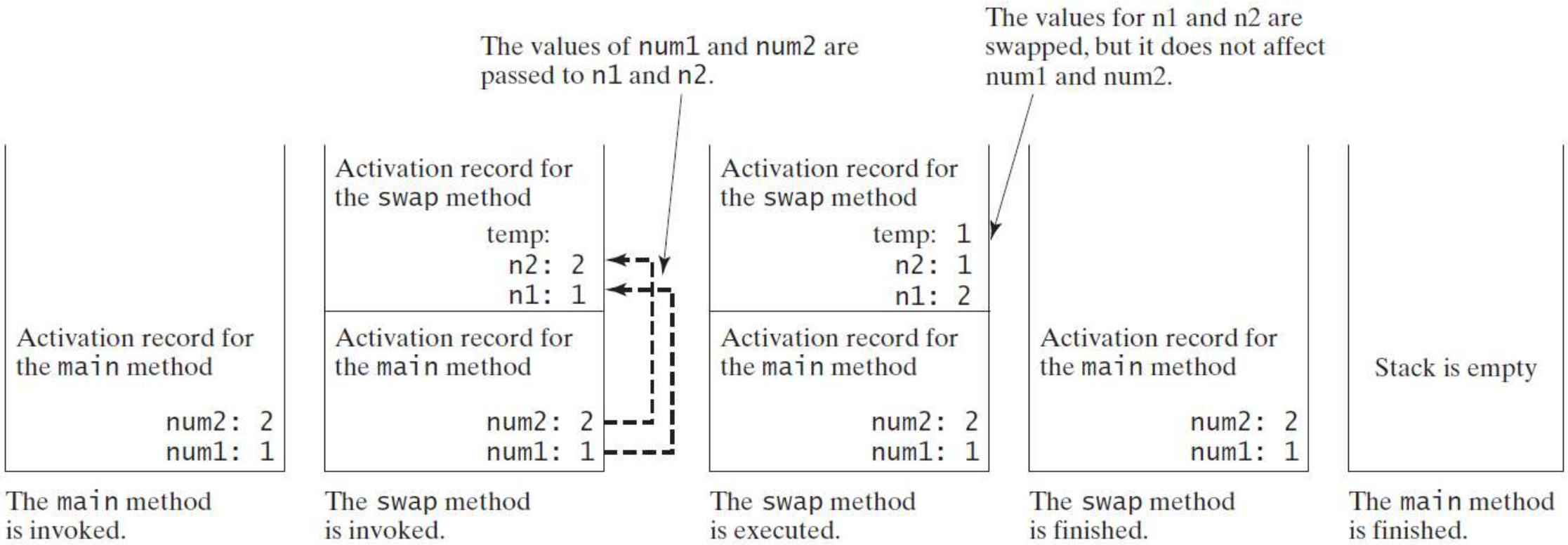
Pass by Value

Testing Pass by value

This program demonstrates passing values to the methods.

▸ Increment ▸ TestPassByValue

Pass by Value, cont.



Modularizing Code

Methods can be used to reduce redundant coding and enable code reuse.

Methods can also be used to modularize code and improve the quality of the program.

▸ GreatestCommonDivisorMethod

▸ PrimeNumberMethod

Overloading Methods

Overloading the **`max`** Method

```
public static double max(double num1, double num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}
```

▸ TestMethodOverloading

Ambiguous Invocation

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));
  }

  public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }

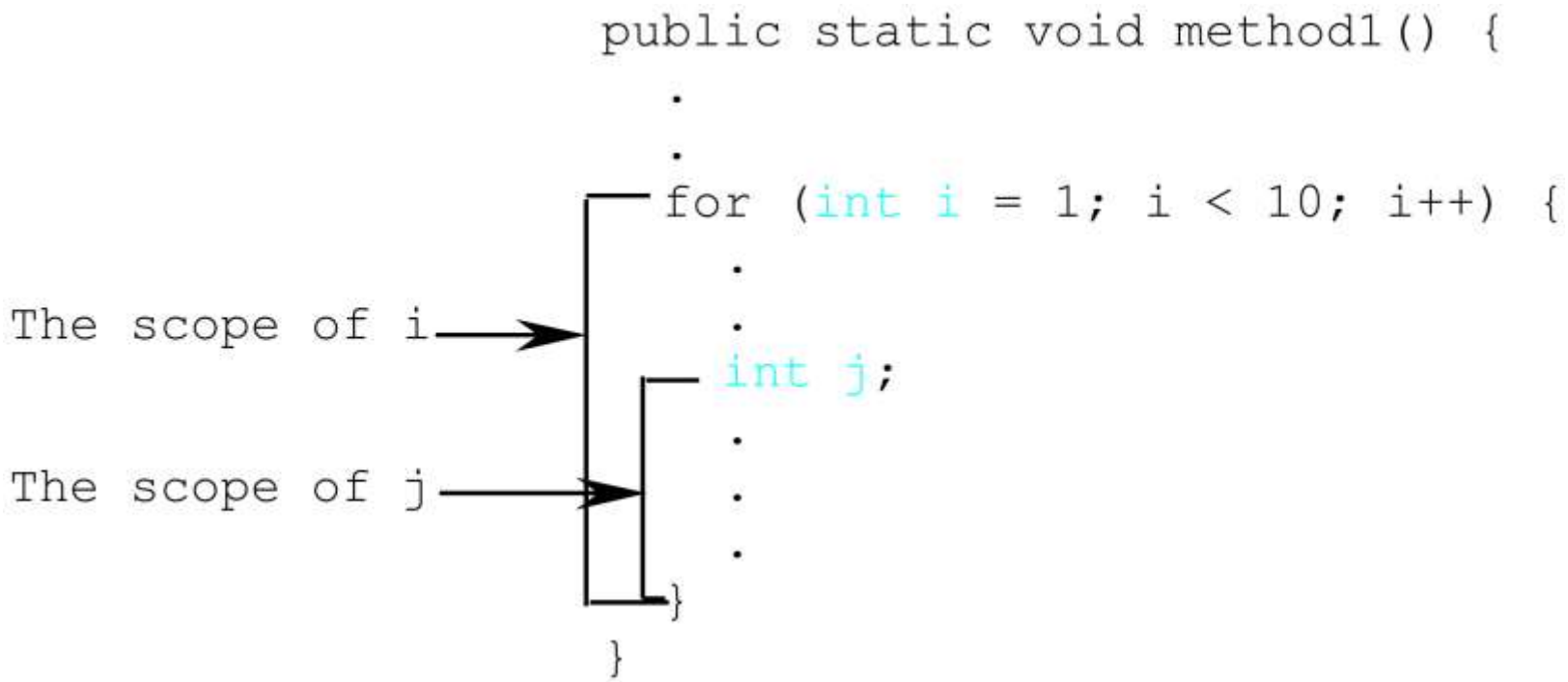
  public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

Scope of Local Variables

- A local variable: a variable defined inside a method.
- Scope: the part of the program where the variable can be referenced.
- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be declared before it can be used.

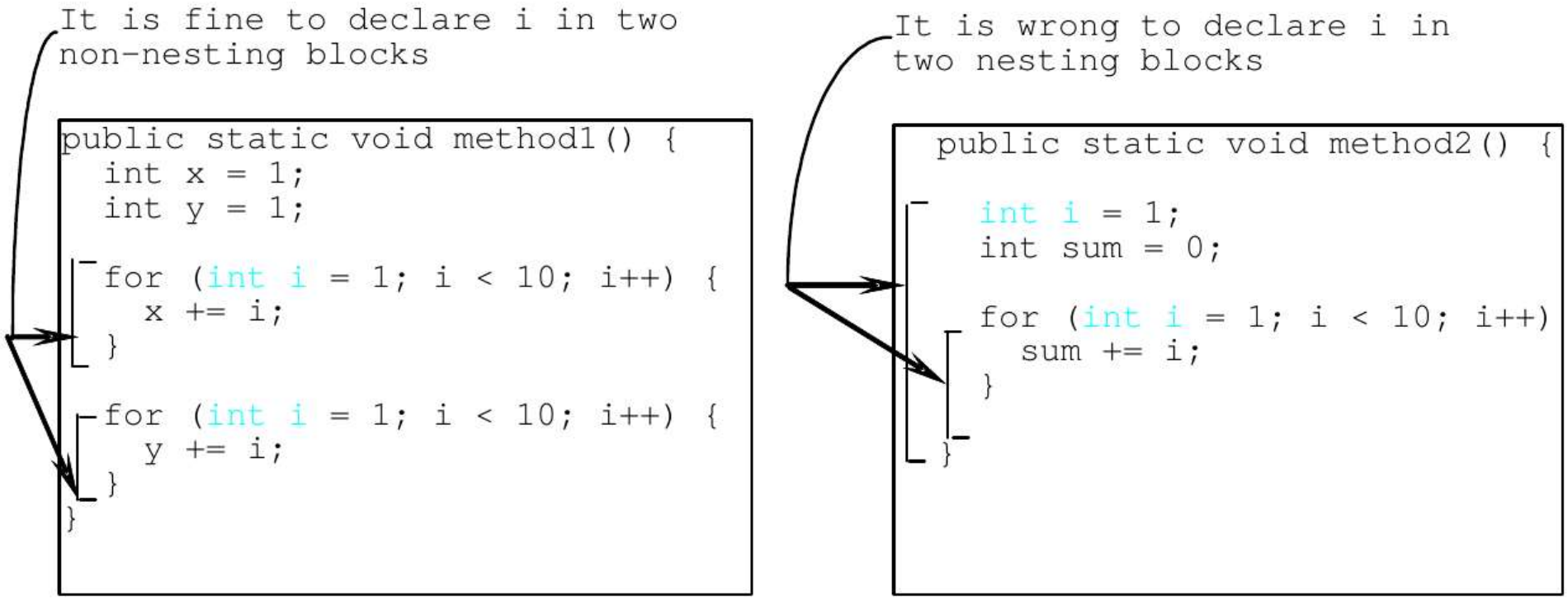
Scope of Local Variables, cont.

A variable declared in the initial action part of a **`for`** loop header has its scope in the entire loop. But a variable declared inside a **`for`** loop body has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.



Scope of Local Variables, cont.

You can declare a local variable with the same name multiple times in different non-nesting blocks in a method, but you cannot declare a local variable twice in nested blocks.



Static Fields

- Static field (class variable): a variable declared static, inside a class but outside any method. Java has no global variables.
- This variable can be accessed by all the methods inside the class.

Static Fields vs Local Variables

```
public class StaticVsLocal {
  static int x=8;

  public static void main(String[] args) {
    System.out.println(x);
    test();
    System.out.println(x);
    t2();
    System.out.println(x);
  }

  public static void test(){
    int x=3;
    ++x;
    System.out.println("Local x: "+x);
  }

  public static void t2(){
    x=7;
    x--;
    System.out.println("Static x: "+x);
  }
}
```

Output

```
8
Local x: 4
8
Static x: 6
6
```

Benefits of Methods

- Write a method once and reuse it anywhere.
- Information hiding. Hide the implementation from the user.
- Reduce complexity.

Java basics

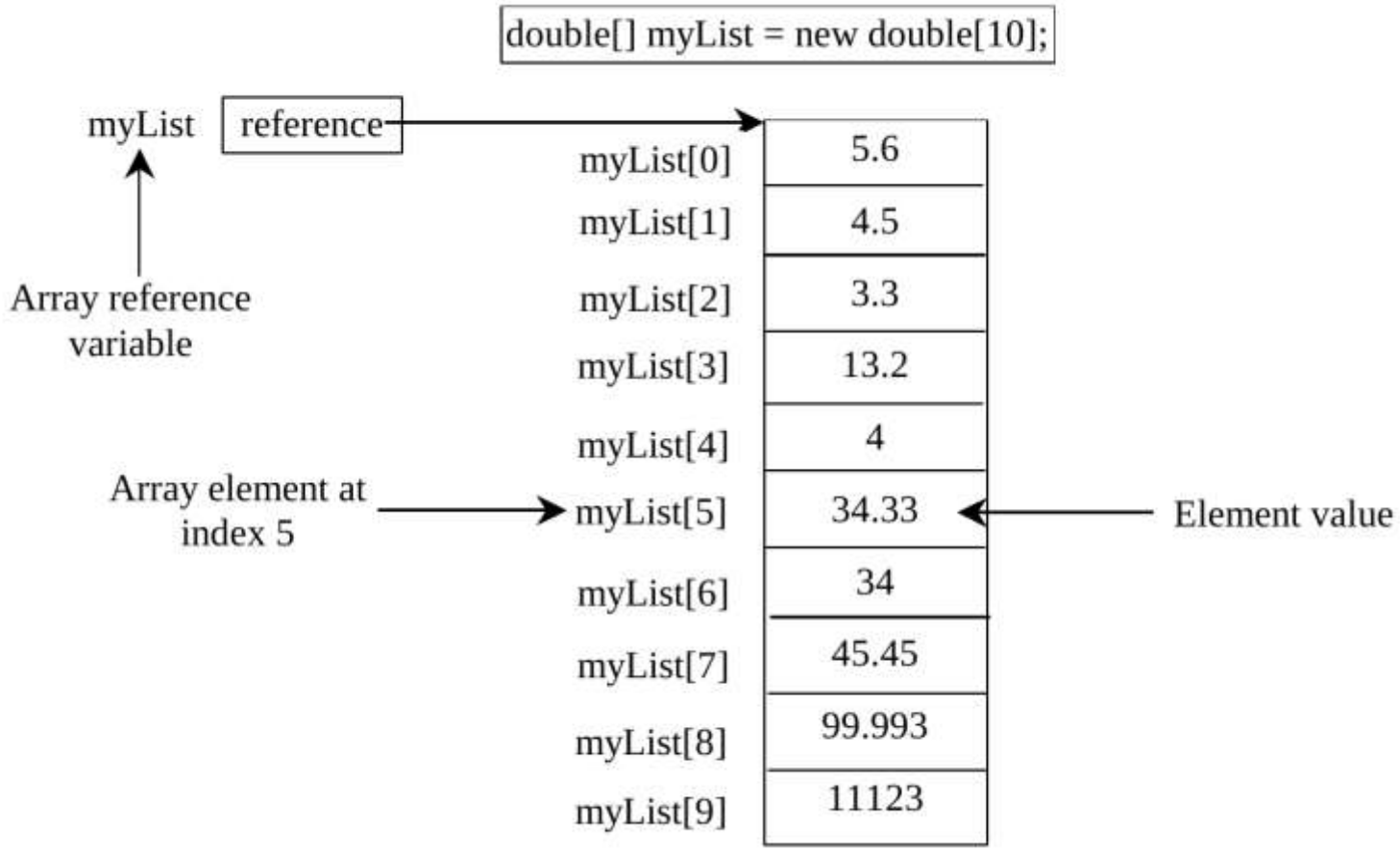
Single-Dimensional Arrays

Opening Problem

Read one hundred numbers, compute their average, and find out how many numbers are above the average.

Introducing Arrays

Array is a data structure that represents a collection of the same types of data.



Declaring Array Variables

```
datatype[] arrayRefVar;
```

Example:

```
double[] myList;
```

```
datatype arrayRefVar[]; // This style is allowed, but not preferred
```

Example:

```
double myList[];
```

Creating Arrays

```
arrayRefVar = new datatype[arraySize];
```

Example:

```
myList = new double[10];
```

`myList[0]` references the first element in the array.

`myList[9]` references the last element in the array.

Declaring and Creating in One Step

```
datatype[] arrayRefVar = new datatype[arraySize];
double[] myList = new double[10];


datatype arrayRefVar[] = new datatype[arraySize];
double myList[] = new double[10];
```

The Length of an Array

Once an array is created, its size is fixed. It cannot be changed. You can find its size using

```
arrayRefVar.length
```

For example,

```
myList.length returns 10
```

Default Values

When an array is created, its elements are assigned the default value of

- 0 for the numeric primitive data types,
- `'\u0000'` for **`char`** types, and
- **`false`** for **`boolean`** types.

Indexed Variables

The array elements are accessed through the index. The array indices are 0-based, i.e., it starts from 0 to **`arrayRefVar.length-1`**.

Each element in the array is represented using the following syntax, known as an indexed variable:

```
arrayRefVar[index];
```

After an array is created, an indexed variable can be used in the same way as a regular variable.
For example, the following code adds the value in **`myList[0]`** and **`myList[1]`** to **`myList[2]`**.

```
myList[2] = myList[0] + myList[1];
```

Array Initializers

Declaring, creating, initializing in one step:

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

This shorthand syntax must be in one statement. This shorthand notation is equivalent to the following statements:

```
double[] myList = new double[4];
myList[0] = 1.9;
myList[1] = 2.9;
myList[2] = 3.4;
myList[3] = 3.5;
```

CAUTION

Using the shorthand notation, you have to declare, create, and initialize the array all in one statement. Splitting it would cause a syntax error. For example, the following is wrong:

```
double[] myList;

myList = {1.9, 2.9, 3.4, 3.5};
```

Processing Arrays

Initializing arrays with input values

```
java.util.Scanner input = new java.util.Scanner(System.in);
System.out.print("Enter " + myList.length + " values: ");
for (int i = 0; i < myList.length; i++)
  myList[i] = input.nextDouble();
```

Initializing arrays with random values

```
for (int i = 0; i < myList.length; i++) {
  myList[i] = Math.random() * 100;
}
```

Printing arrays

```
for (int i = 0; i < myList.length; i++) {
  System.out.print(myList[i] + " ");
}
```

Processing Arrays

Summing all elements

```
double total = 0;
for (int i = 0; i < myList.length; i++) {
  total += myList[i];
}
```

Finding the largest element

```
double max = myList[0];
for (int i = 1; i < myList.length; i++) {
  if (myList[i] > max) max = myList[i];
}
```

Enhanced for Loop (for-each loop)

JDK 1.5 introduced a new for loop that enables you to traverse the complete array sequentially without using an index variable. For example, the following code displays all elements in the array **`myList`**:

```
for (double value: myList)
  System.out.println(value);
```

In general, the syntax is

```
for (elementType value: arrayRefVar) {
  // Process the value
}
```

You still have to use an index variable if you wish to traverse the array in a different order or change the elements in the array.

Copying Arrays

Using a loop:

```
int[] sourceArray = {2, 3, 1, 5, 10};
int[] targetArray = new int[sourceArray.length];

for (int i = 0; i < sourceArray.length; i++)
  targetArray[i] = sourceArray[i];
```

The `arraycopy` Utility

```
arraycopy(sourceArray, src_pos, targetArray, tar_pos, length);
```

Example:

```
System.arraycopy(sourceArray, 0, targetArray, 0, sourceArray.length);
```

Passing Arrays to Methods

```
public static void printArray(int[] array) {
  for (int i = 0; i < array.length; i++) {
    System.out.print(array[i] + " ");
  }
}
```

Invoke the method

```
int[] list = {3, 1, 2, 6, 4, 2};
printArray(list);
```

Invoke the method

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

Anonymous array

Anonymous Array

The statement

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

creates an array using the following syntax:

```
new dataType[]{literal0, literal1, ..., literalk};
```

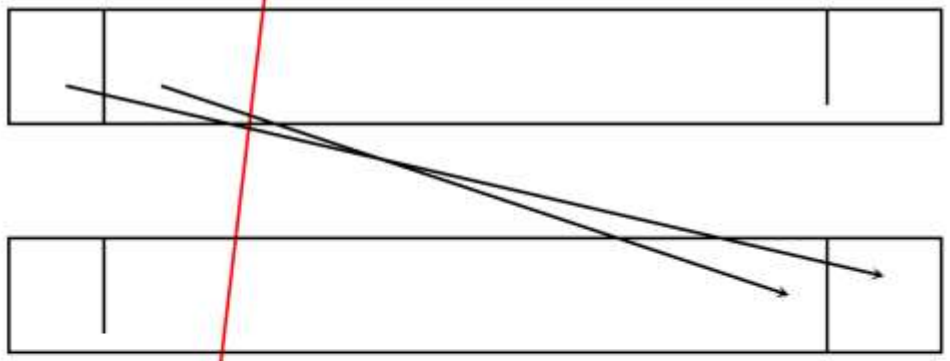
There is no explicit reference variable for the array. Such array is called an anonymous array.

Returning an Array from a Method

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

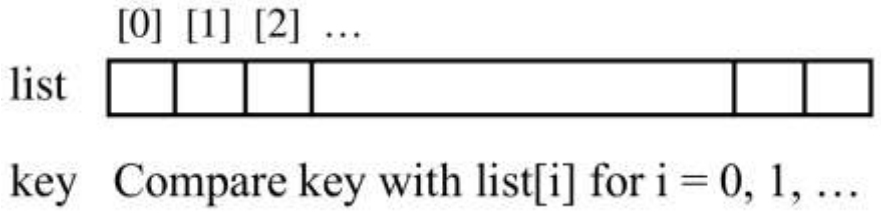



```
int[] list1 = {1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

Searching Arrays

Searching is the process of looking for a specific element in an array; for example, discovering whether a certain score is included in a list of scores. Searching is a common task in computer programming. There are many algorithms and data structures devoted to searching. In this section, two commonly used approaches are discussed, linear search and binary search.

```
public class LinearSearch {
  /** The method for finding a key in the list */
  public static int linearSearch(int[] list, int key) {
    for (int i = 0; i < list.length; i++)
      if (key == list[i])
        return i;
    return -1;
  }
}
```



The `Arrays.binarySearch` Method

Since binary search is frequently used in programming, Java provides several overloaded **`binarySearch`** methods for searching a key in an array of **`int`**, **`double`**, **`char`**, **`short`**, **`long`**, and **`float`** in the **`java.util.Arrays`** class. For example, the following code searches the keys in an array of numbers and an array of characters.

```
int[] list = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};
System.out.println("Index is " +
  java.util.Arrays.binarySearch(list, 11));

char[] chars = {'a', 'c', 'g', 'x', 'y', 'z'};
System.out.println("Index is " +
  java.util.Arrays.binarySearch(chars, 't'));
```

For the **`binarySearch`** method to work, the array must be pre-sorted in increasing order.

The `Arrays.sort` Method

Since sorting is frequently used in programming, Java provides several overloaded **`sort`** methods for sorting an array of **`int`**, **`double`**, **`char`**, **`short`**, **`long`**, and **`float`** in the **`java.util.Arrays`** class. For example, the following code sorts an array of numbers and an array of characters.

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
java.util.Arrays.sort(numbers);

char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(chars);
```

`Arrays.parallelSort(list)` utilizes the multicore for fast sorting.

Java basics

Multidimensional Arrays

Declare/Create Two-dimensional Arrays

```
// Declare array ref var
dataType[][] refVar;

// Create array and assign its reference to variable
refVar = new dataType[10][10];

// Combine declaration and creation in one statement
dataType[][] refVar = new dataType[10][10];

// Alternative syntax
dataType refVar[][] = new dataType[10][10];
```

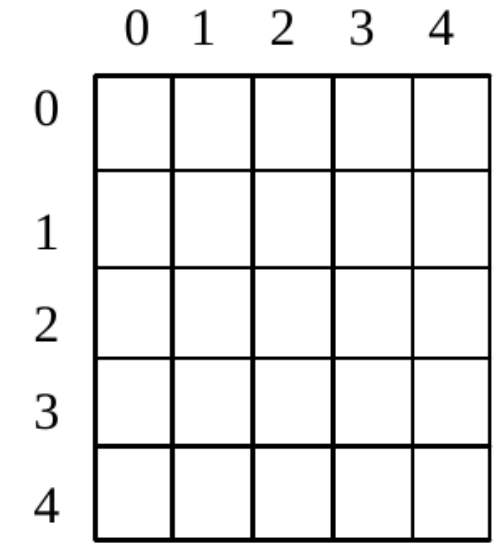
Declaring and Creating Two-dimensional Arrays

```
int[][] matrix = new int[10][10];
 or
int matrix[][] = new int[10][10];
matrix[0][0] = 3;

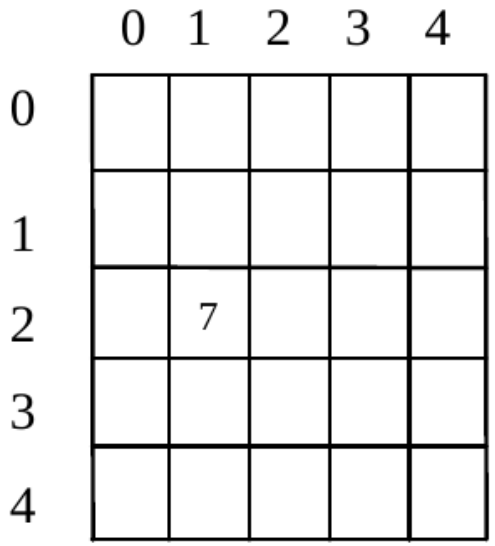
for (int i = 0; i < matrix.length; i++)
  for (int j = 0; j < matrix[i].length; j++)
    matrix[i][j] = (int)(Math.random() * 1000);

double[][] x;
```

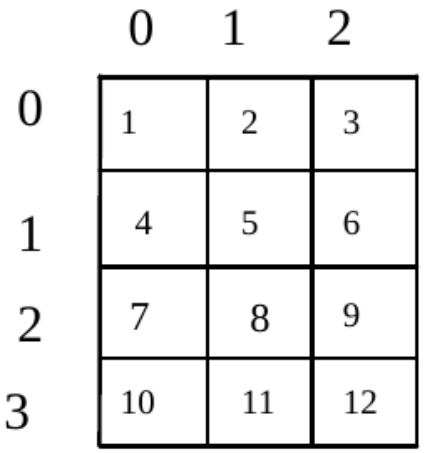
Two-dimensional Array Illustration




```
matrix = new int[5][5];
```




```
matrix[2][1] = 7;
```




```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

matrix.length? 5
matrix[0].length? 5

array.length? 4
array[0].length? 3

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an array initializer to declare, create and initialize a two-dimensional array. For example,

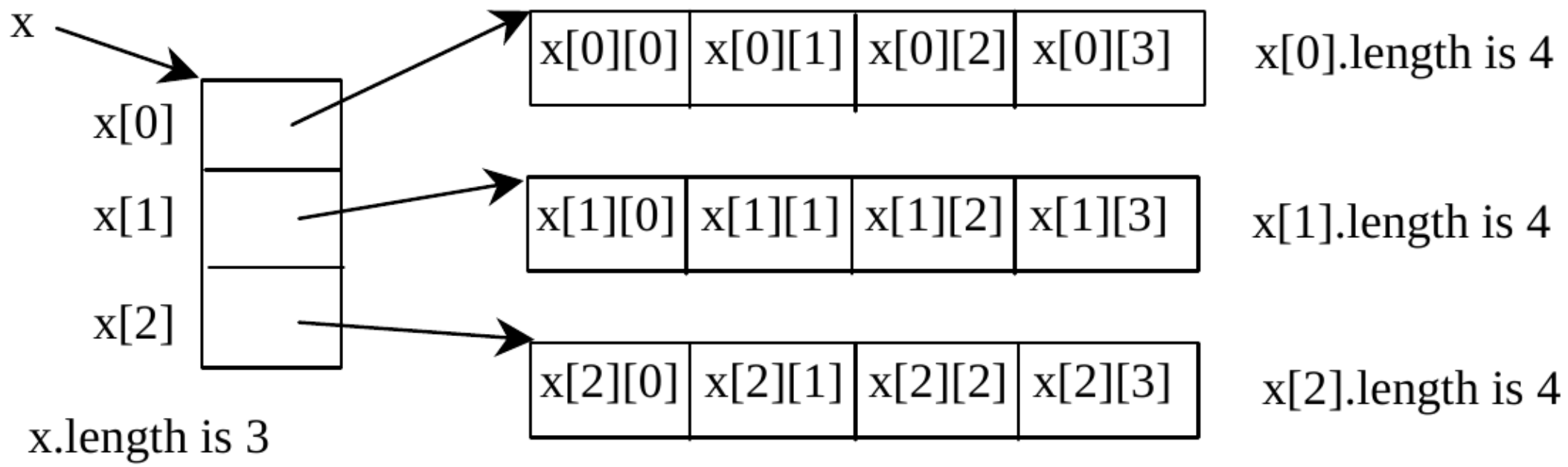
```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

Same as

```
int[][] array = new int[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

Lengths of Two-dimensional Arrays

```
int[][] x = new int[3][4];
```



Lengths of Two-dimensional Arrays, cont.

```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

```
array.length
array[0].length
array[1].length
array[2].length
array[3].length
```

```
array[4].length
```

`ArrayIndexOutOfBoundsException`

Ragged Arrays

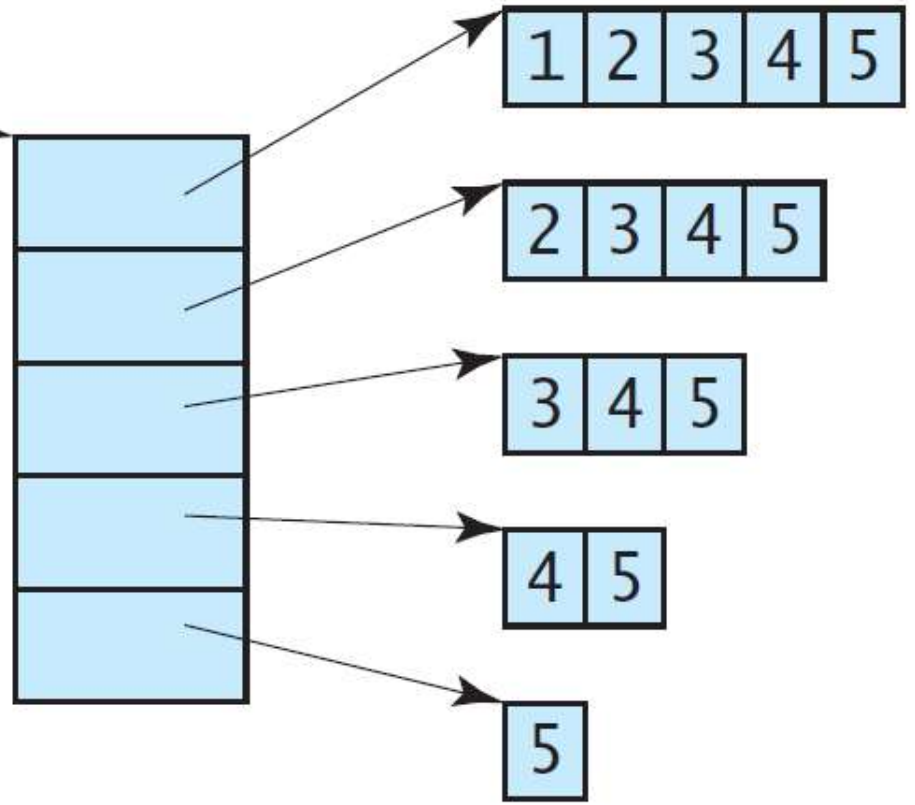
Each row in a two-dimensional array is itself an array. So, the rows can have different lengths. Such an array is known as a ragged array. For example,

```
int[][] matrix = {
  {1, 2, 3, 4, 5},
  {2, 3, 4, 5},
  {3, 4, 5},
  {4, 5},
  {5}
};
```

```
matrix.length is 5
matrix[0].length is 5
matrix[1].length is 4
matrix[2].length is 3
matrix[3].length is 2
matrix[4].length is 1
```

Ragged Arrays, cont.

```
int[][] triangleArray = {
   {1, 2, 3, 4, 5},
   {2, 3, 4, 5},
   {3, 4, 5},
   {4, 5},
   {5}
};
```



Multidimensional Arrays

Occasionally, you will need to represent n-dimensional data structures. In Java, you can create n-dimensional arrays for any integer n.

The way to declare two-dimensional array variables and create two-dimensional arrays can be generalized to declare n-dimensional array variables and create n-dimensional arrays for n >= 3. For example, the following syntax declares a three-dimensional array variable **`scores`**, creates an array, and assigns its reference to **`scores`**.

```
double[][][] scores = new double[10][5][2];
```

Java basics

Recursion

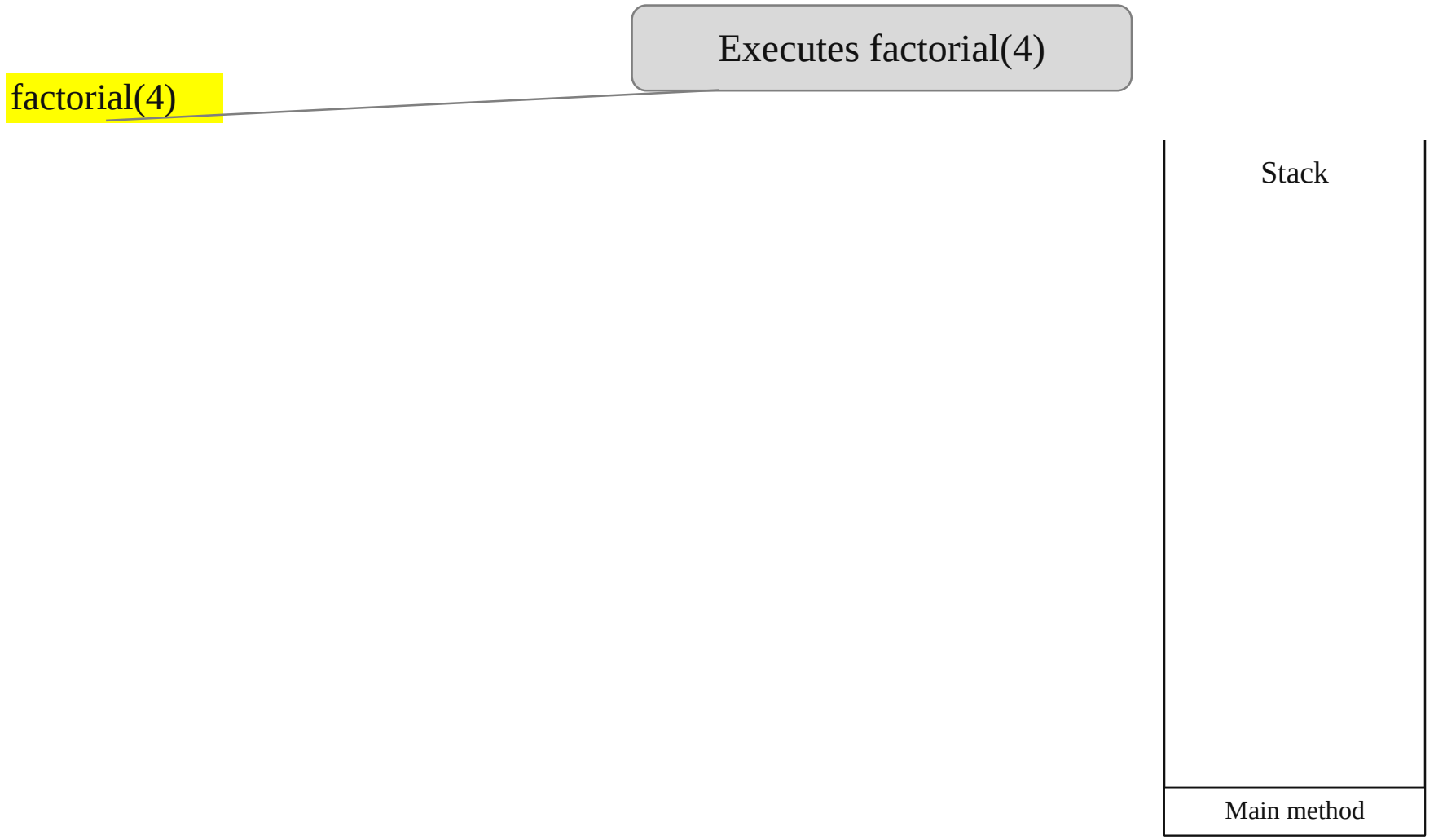
Computing Factorial

```
factorial(0) = 1;
factorial(n) = n*factorial(n-1);
```

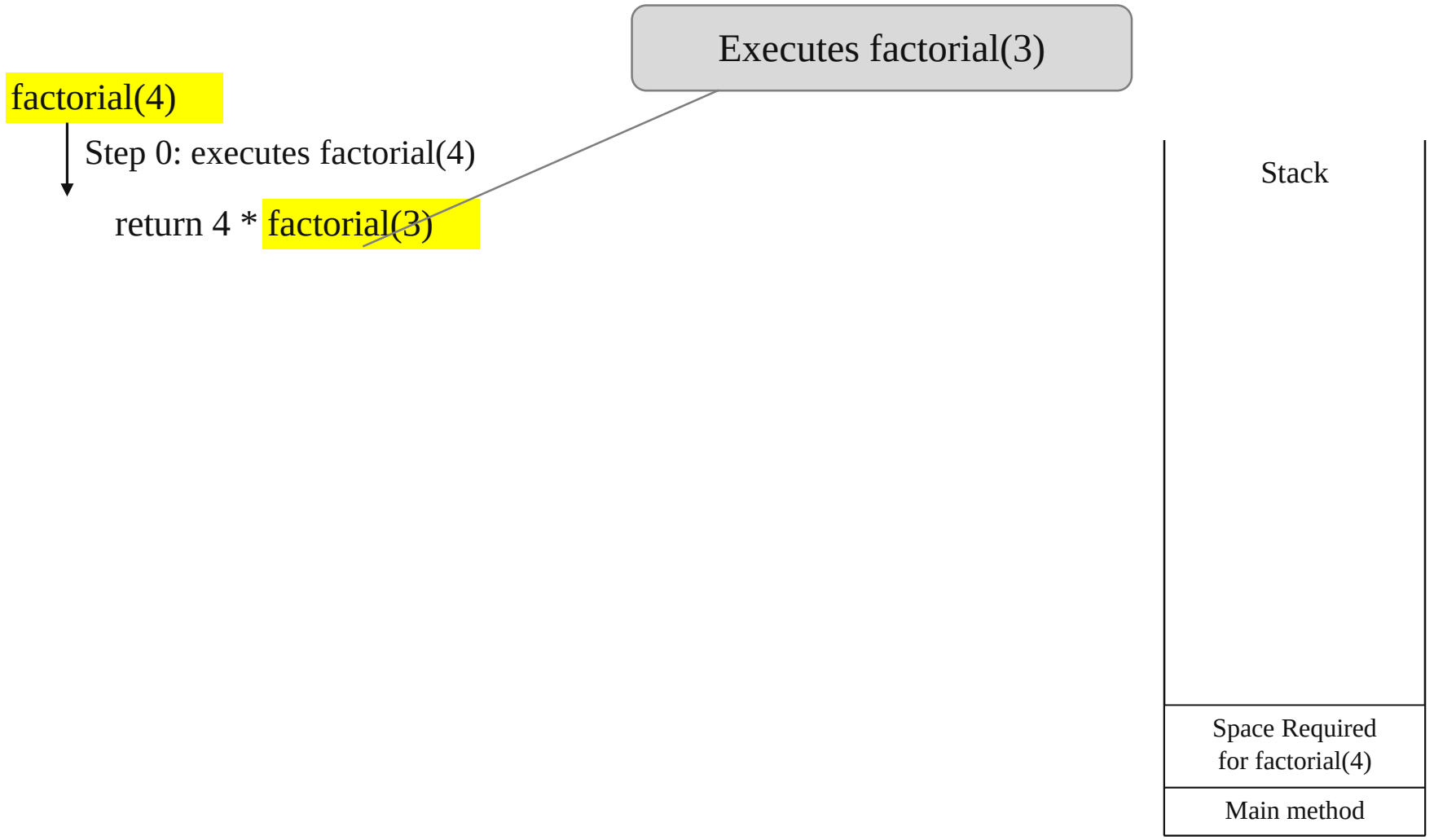
```
n! = n * (n-1)!
```

▸ ComputeFactorial

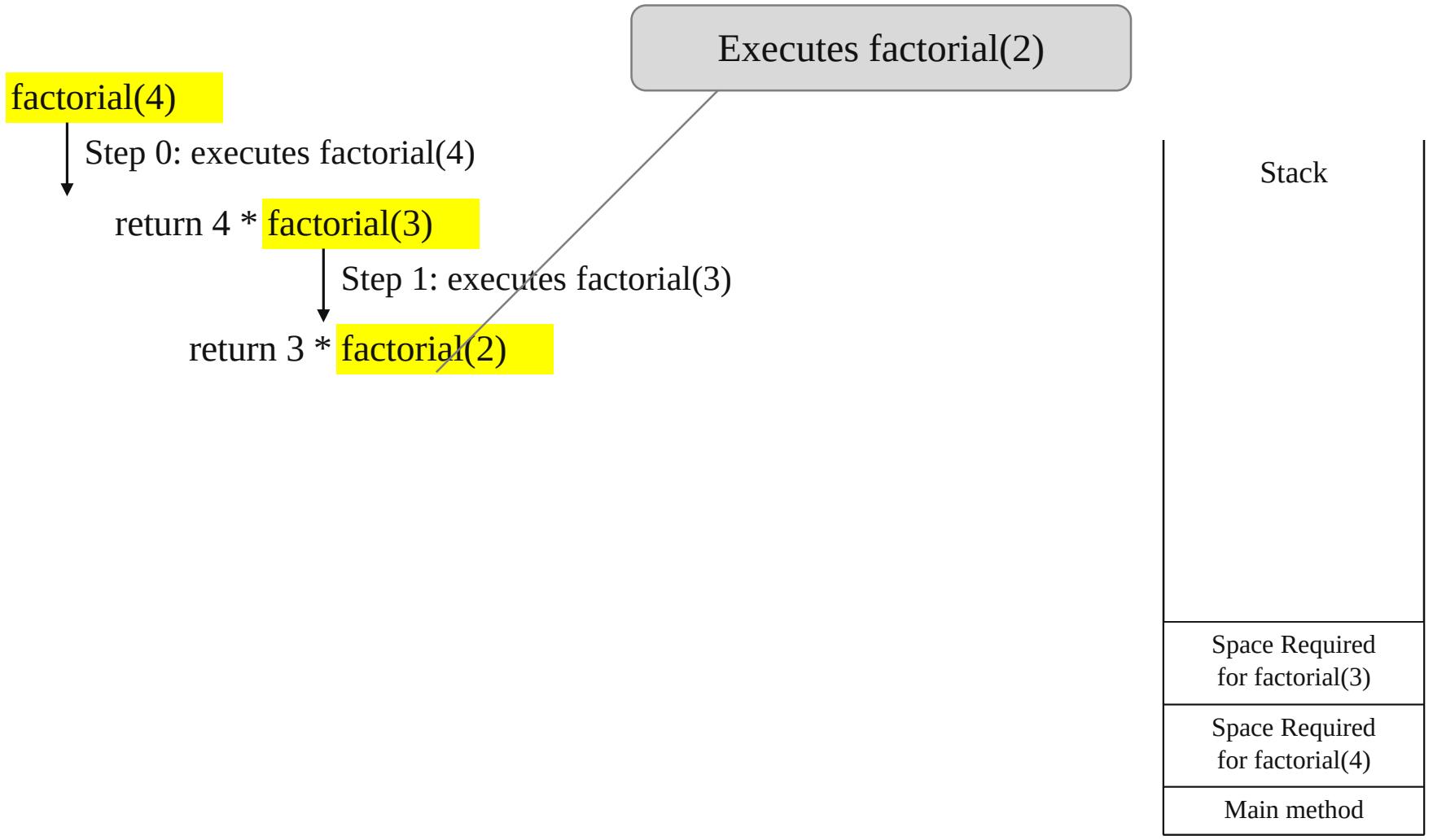
Trace Recursive factorial



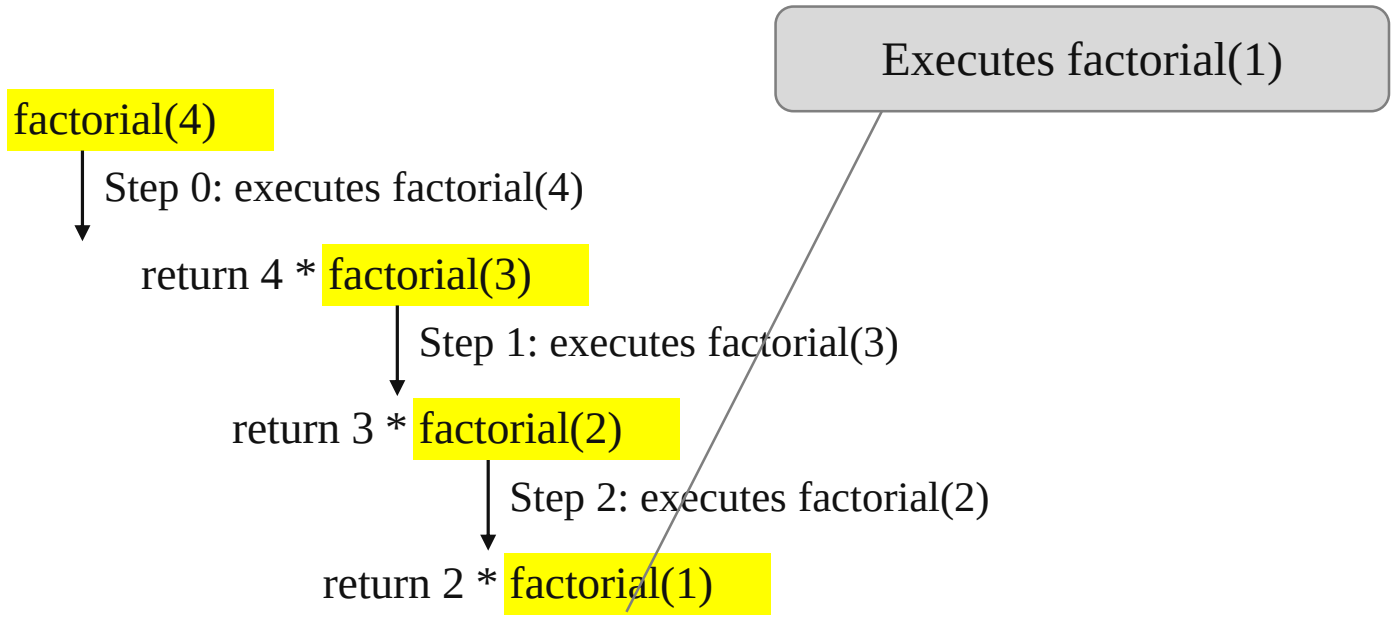
Trace Recursive factorial

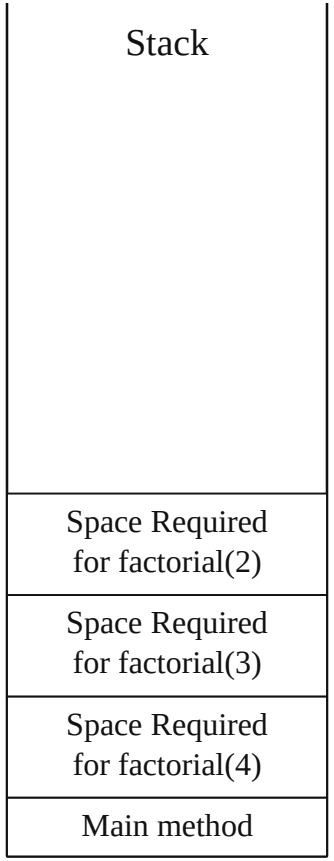


Trace Recursive factorial

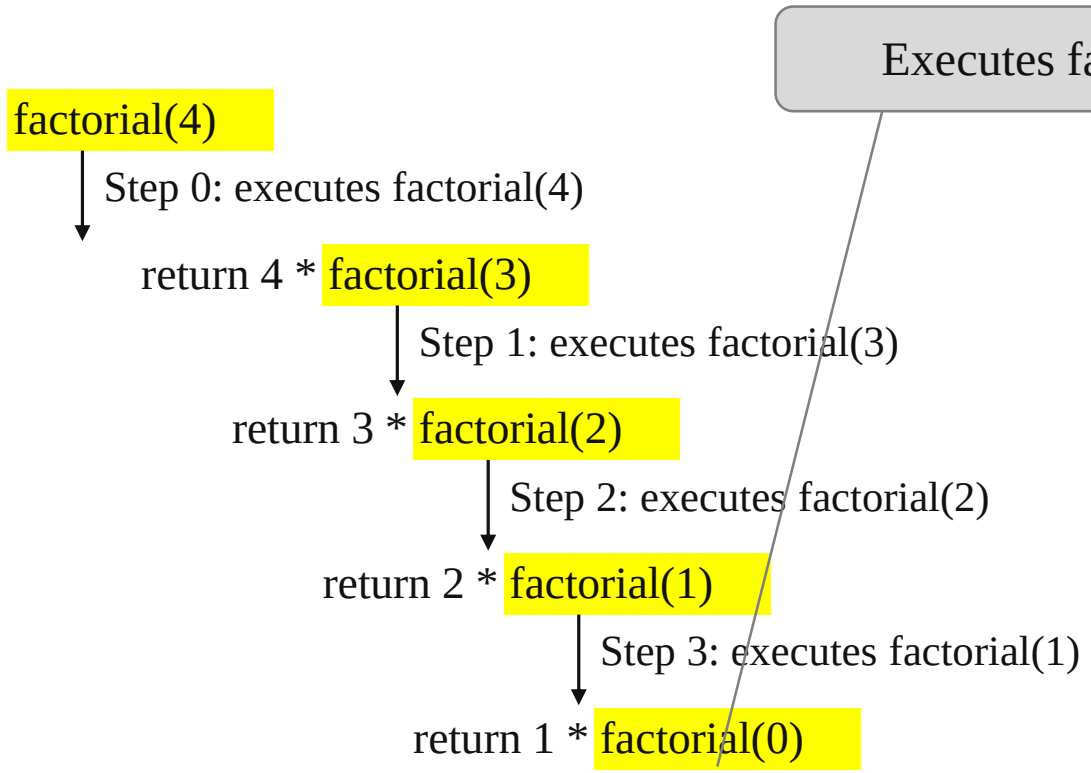


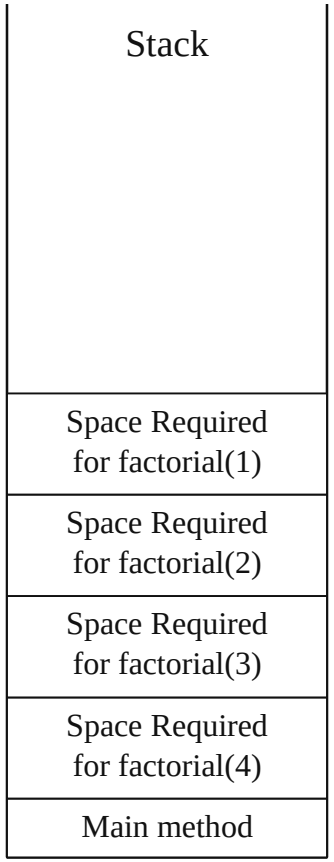
Trace Recursive factorial



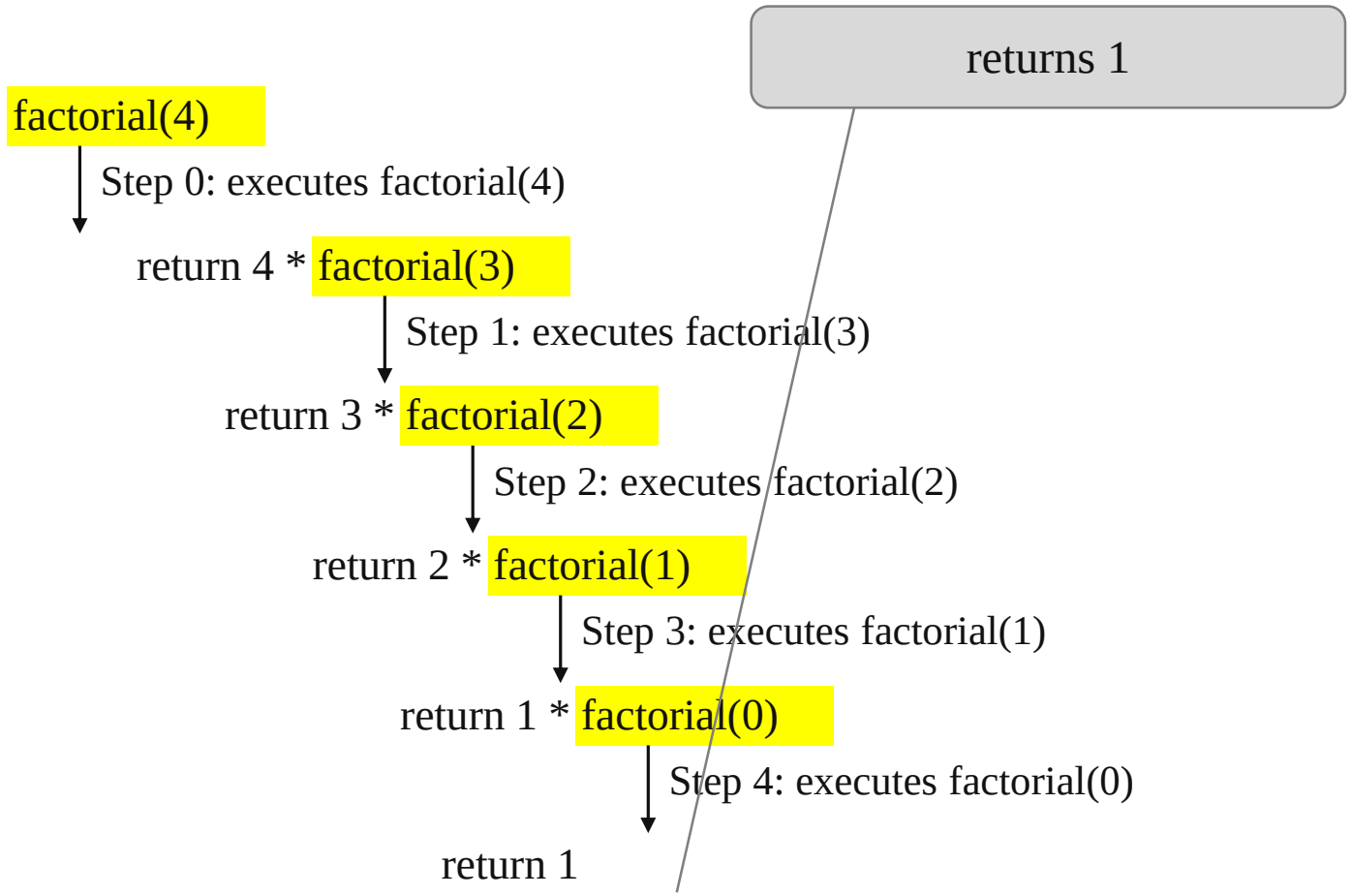


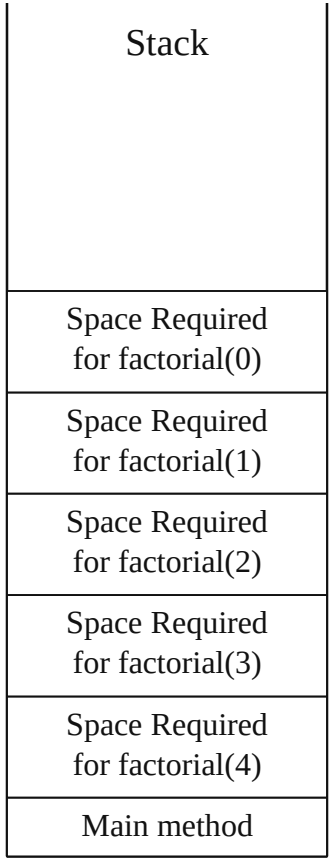
Trace Recursive factorial



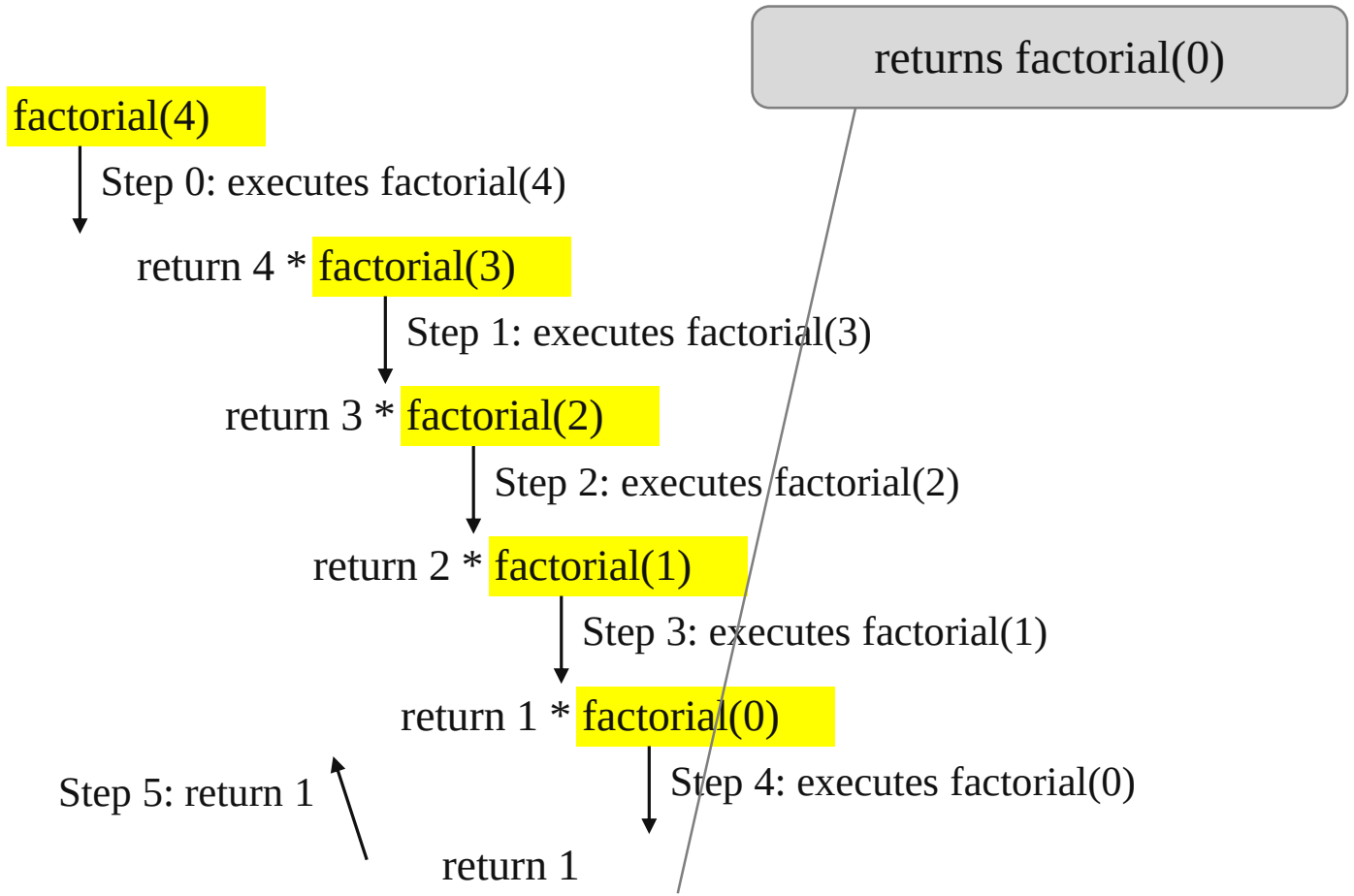


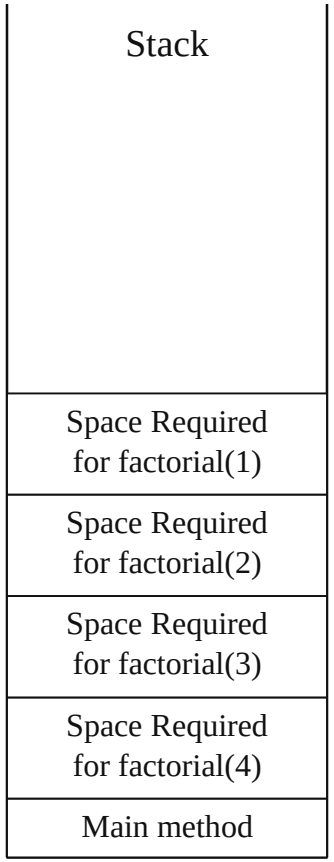
Trace Recursive factorial



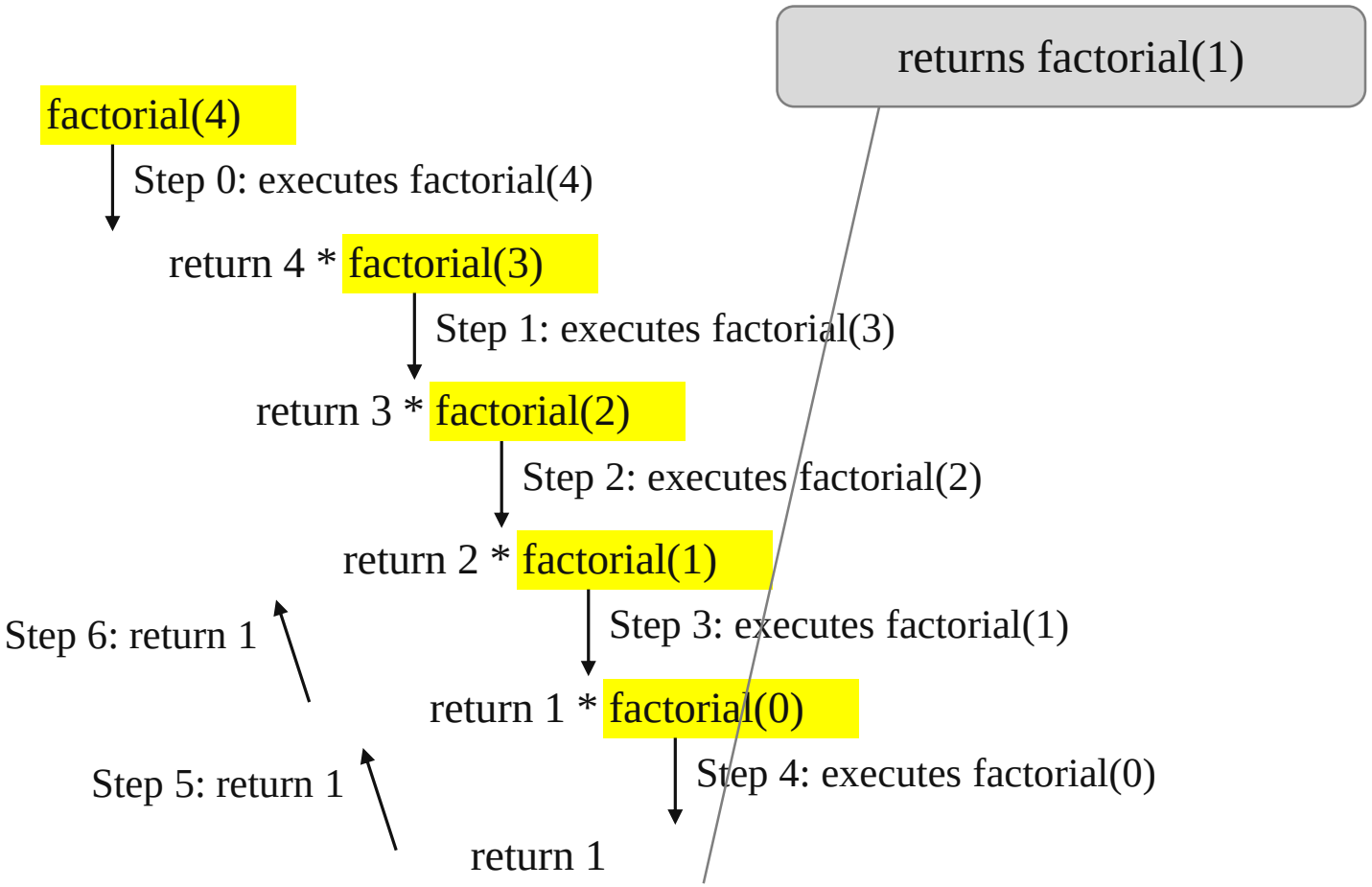


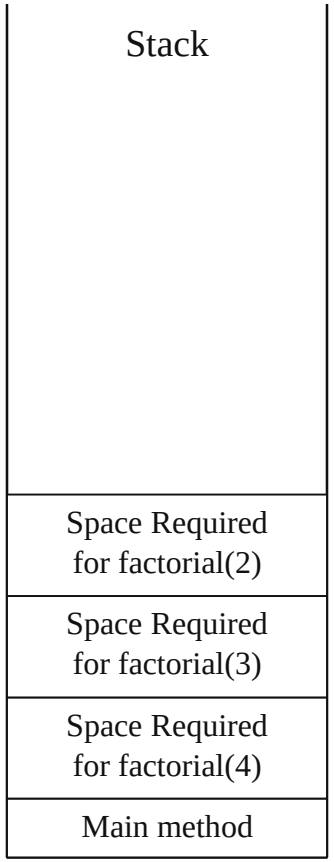
Trace Recursive factorial



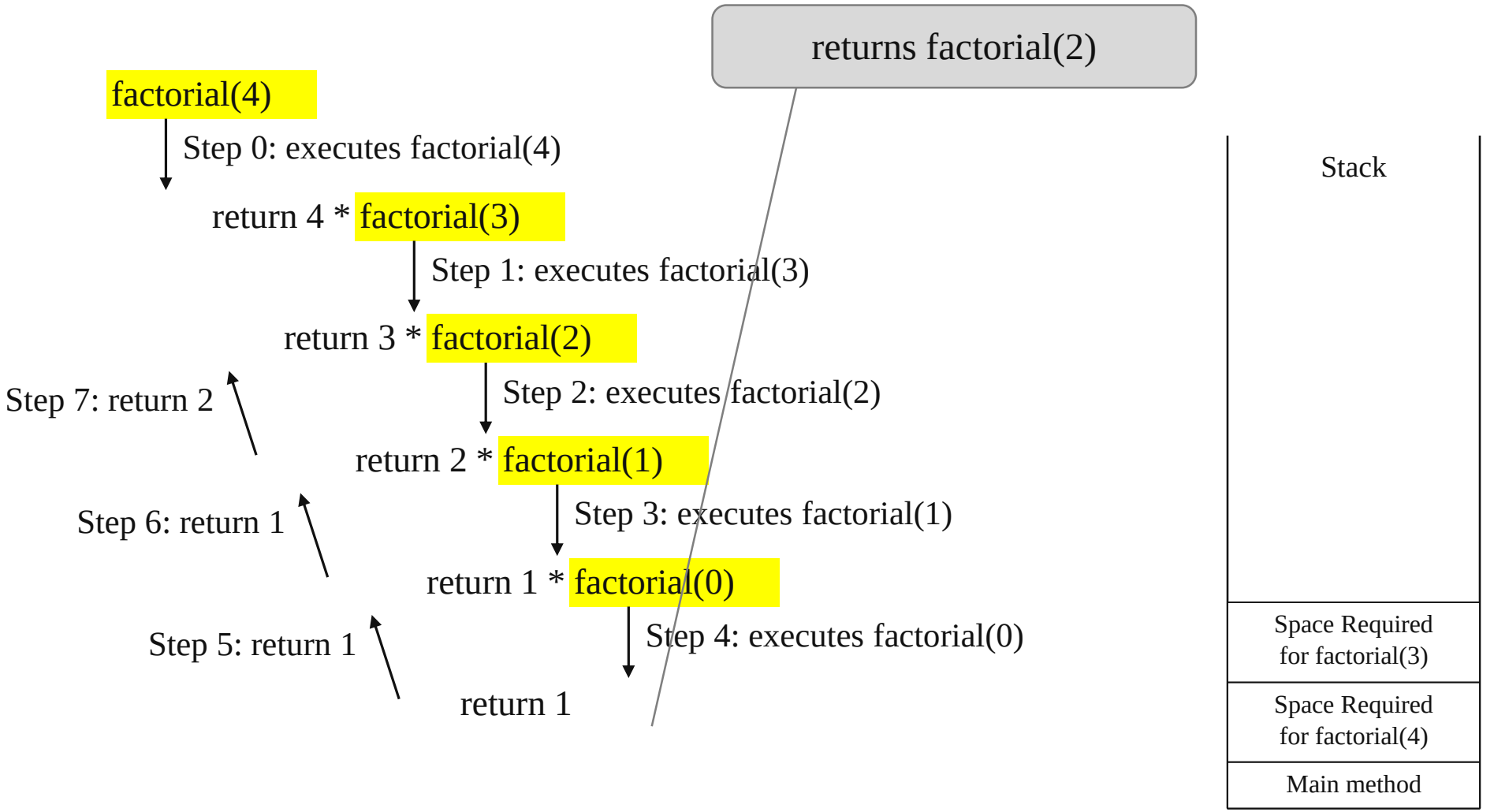


Trace Recursive factorial

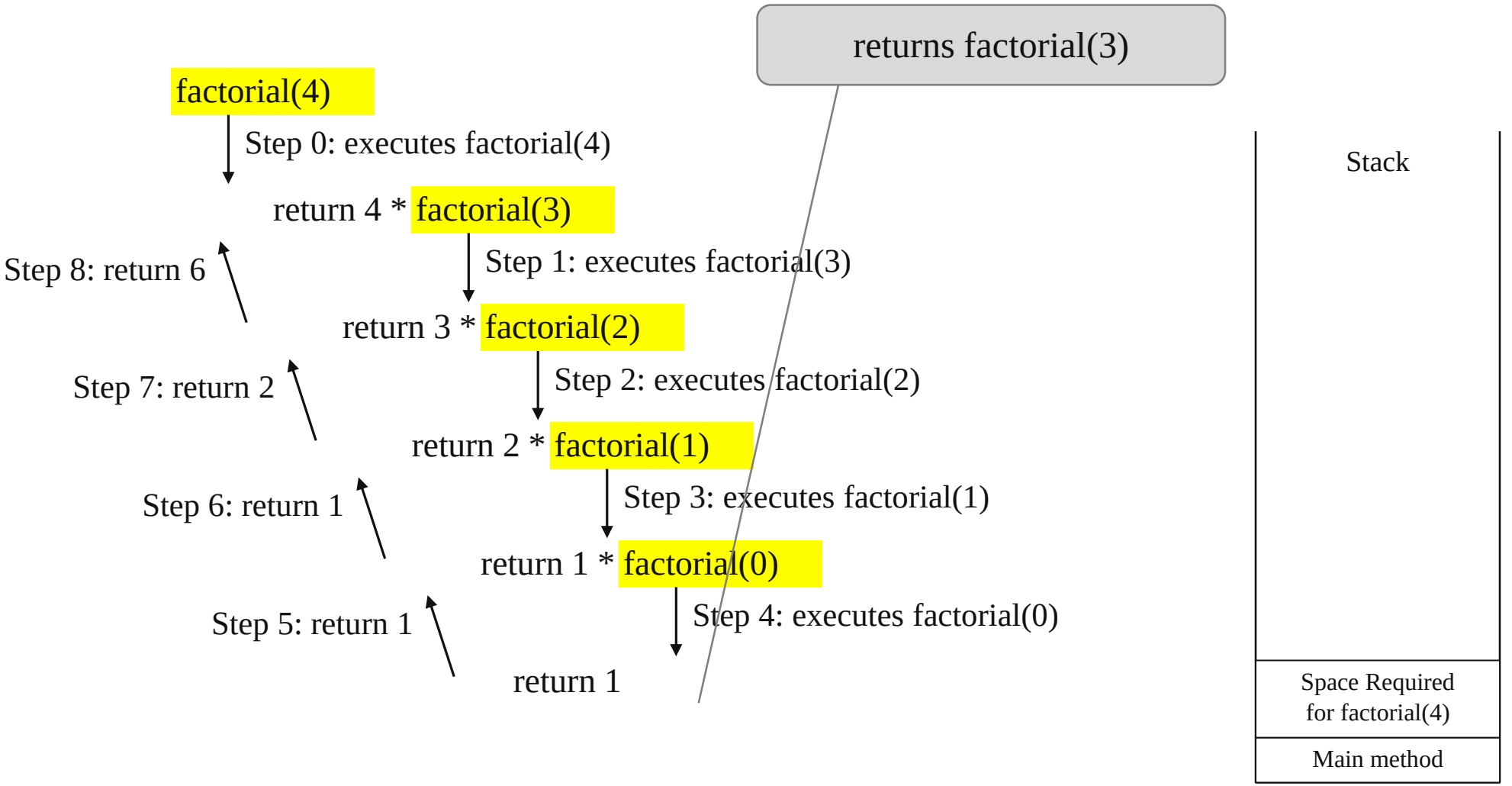




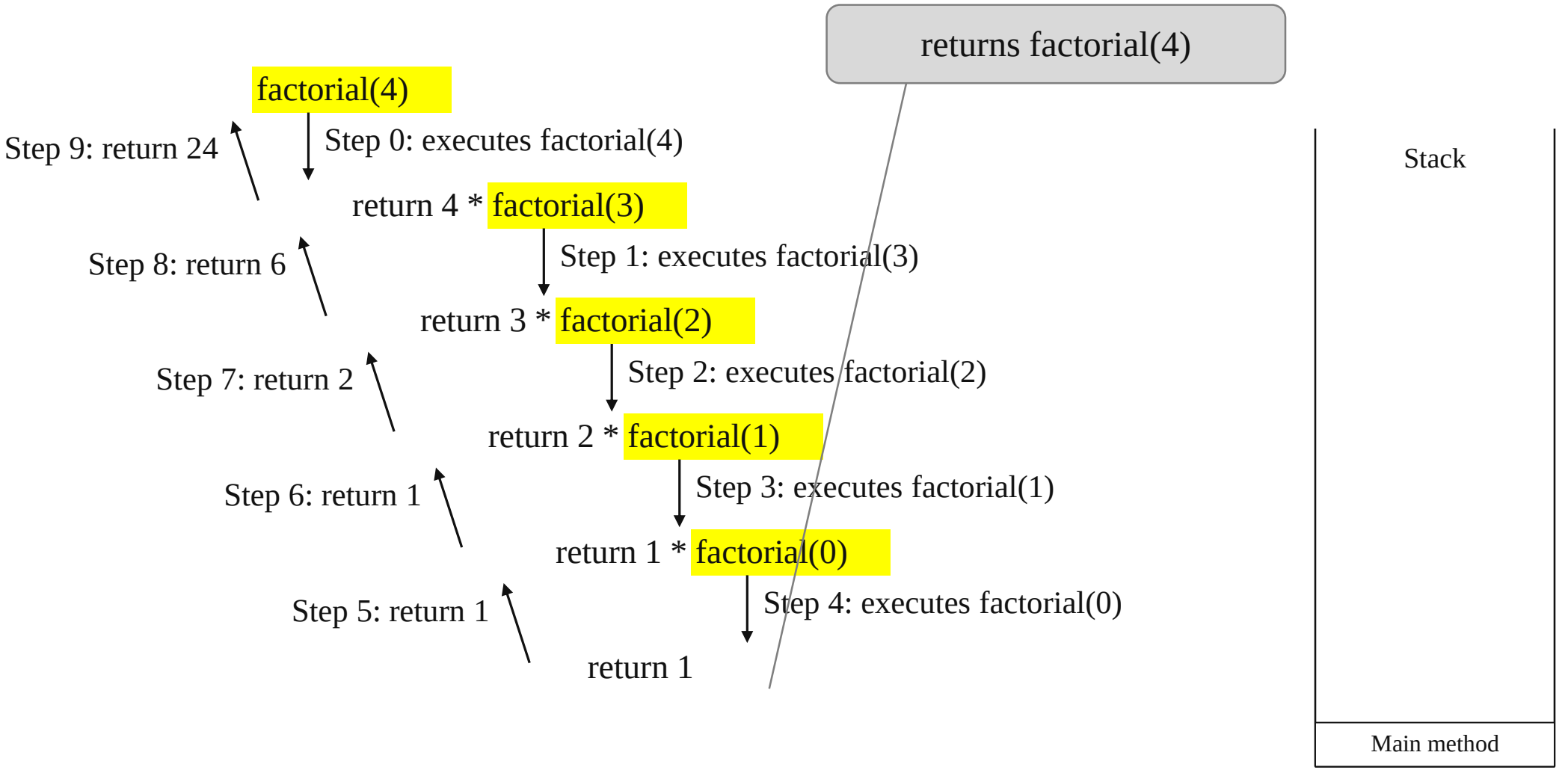
Trace Recursive factorial



Trace Recursive factorial



Trace Recursive factorial



Fibonacci Numbers

```
Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89...
         indices: 0 1 2 3 4 5 6 7  8  9  10 11

fib(0) = 0;
fib(1) = 1;
fib(index) = fib(index -1) + fib(index -2); index >=2


fib(3) = fib(2) + fib(1) = (fib(1) + fib(0)) + fib(1) =
         (1 + 0) + fib(1) = 1 + fib(1) = 1 + 1 = 2
```

▸ ComputeFibonacci

Characteristics of Recursion

All recursive methods have the following characteristics:

- One or more base cases (the simplest case) are used to stop recursion.
- Every recursive call reduces the original problem, bringing it increasingly closer to a base case until it becomes that case.

In general, to solve a problem using recursion, you **break it into subproblems**.

If a subproblem resembles the original problem, you can apply the same approach to solve the subproblem recursively. This subproblem is almost the same as the original problem in nature with a smaller size.

Java basics

Arguments

Using Command-Line Arguments

- Command-line arguments
  - Pass arguments from the command line

```
String args[]
```

  - Appear after the class name in the java command

```
java MyClass a b
```

  - Number of arguments passed in from command line

```
args.length
```

  - First command-line argument

```
args[ 0 ]
```

Using Command-Line Arguments

```
C:\javaex\> java sum 5 6 8 -7 13

public class test {
  publis static void main(String[] args){
    sum=0;
    System.out.print("The sum of ");
    for(int i:args) {
      sum+=i;
      System.out.print(i+",");
    }
    System.out.print(" is "+sum+"\n");
  }
}
```

Output

```
The sum of 5, 6, 8, -7, 13 is 25
```

Variable-Length Argument Lists

- Variable-length argument lists
  - Unspecified number of arguments
  - Use ellipsis (…) in method's parameter list
  - Can occur only once in parameter list
  - Must be placed at the end of parameter list
  - Array whose elements are all of the same type

Variable-Length Argument Lists

```
public class VarargsTest {
   public static double average( double... numbers ) {
      double total = 0.0;
      for ( double d : numbers )
         total += d;
      return total / numbers.length;
   }
   public static void main( String[] args ) {
      double d1 = 10.0;
      double d2 = 20.0;
      double d3 = 30.0;
      double d4 = 40.0;
      System.out.printf( "d1 = %.1f\nd2 = %.1f\nd3 = %.1f\nd4 = %.1f\n\n", d1, d2, d3, d4 );
      System.out.printf( "Average of d1 and d2 is %.1f\n",  average( d1, d2 ) );
      System.out.printf( "Average of d1, d2 and d3 is %.1f\n",  average( d1, d2, d3 ) );
      System.out.printf( "Average of d1, d2, d3 and d4 is %.1f\n",  average( d1, d2, d3, d4 ) );
   }
}
```